



From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows

Mohamed Amine Ferrag^{a,*}, Norbert Tihanyi^{b,c}, Djallel Hamouda^d, Leandros Maglaras^e, Abderrahmane Lakas^a, Merouane Debbah^f

^a Department of Computer and Network Engineering, College of Information Technology, United Arab Emirates University, Al Ain, United Arab Emirates

^b Technology Innovation Institute, Abu Dhabi, United Arab Emirates

^c Eötvös Loránd University, Hungary

^d Department of Computer Science, Guelma University, Algeria

^e De Montfort University, Leicester, United Kingdom

^f Khalifa University of Science and Technology, Abu Dhabi, United Arab Emirates

ARTICLE INFO

Keywords:

Security
Large language models
Autonomous AI agents
Agentic AI
Reasoning

ABSTRACT

Autonomous AI agents powered by large language models (LLMs) with structured function-calling interfaces have greatly expanded capabilities for real-time data retrieval, computation, and multi-step orchestration. However, the rapid growth of plugins, connectors, and inter-agent protocols has outpaced security practices, leading to brittle integrations — plugin APIs and protocol adapters that rely on ad-hoc authentication, inconsistent schemas, and weak validation — making them vulnerable to failures and exploitation. This survey introduces a unified end-to-end threat model for LLM-agent ecosystems, spanning host-to-tool and agent-to-agent communications, and catalogs over thirty attack techniques across Input Manipulation, Model Compromise, System and Privacy Attacks, and Protocol Vulnerabilities. For each category, we provide a formal mathematical formulation of the underlying threat model, defining attacker capabilities, objectives, and affected layers to enable systematic analysis. Representative examples include Prompt-to-SQL (P2SQL) injections and the Toxic Agent Flow exploit in GitHub's MCP server. For each category, we assess feasibility, review defenses, and outline mitigation strategies such as dynamic trust management, cryptographic provenance tracking, and sandboxed agentic interfaces. The framework was validated through expert review and cross-mapping with real-world incidents and public vulnerability repositories (e.g., CVE, NIST NVD) to ensure practical relevance. Compared to prior surveys, this work provides the first integrated taxonomy bridging input-level exploits and protocol-layer vulnerabilities in LLM-agent ecosystems while introducing formal system definitions for each threat class. Ultimately, it offers actionable insights for securing next-generation AI agents through layered defense and continuous verification. Our work provides a comprehensive reference to guide the design of secure and resilient LLM-agent workflows.

1. Introduction

Autonomous AI agents have advanced rapidly in the past two years, driven by large language models (LLMs) that can perceive, reason, and act in complex environments. The introduction of structured function calling in 2023 enabled LLMs to invoke external APIs in a consistent, machine-readable format, allowing them to fetch real-time data, perform computations, and orchestrate multi-step workflows [1–3]. At the same time, a vibrant ecosystem of plugins and extension frameworks emerged. ChatGPT plugins provided turnkey access to third-party services; frameworks such as LangChain and LlamaIndex offered standardized connector interfaces; and dedicated AI app stores began

to host curated collections of tools [4,5]. Together, these developments have greatly expanded the practical capabilities of autonomous agents [6–9]. Despite this progress, tool integration remains siloed and labor-intensive. Developers must write bespoke adapter code for each new service, manually manage credentials and error handling, and learn platform-specific conventions for function calls. Workflows are often hard-coded, which limits an agent's ability to adapt at runtime and makes extension cumbersome. Moreover, the absence of a shared discovery mechanism or common vocabulary for tool capabilities increases the risk of security gaps when onboarding new services or updating existing ones [10–14].

* Corresponding author.

E-mail address: mohamed.ferrag@uaeu.ac.ae (M.A. Ferrag).

<https://doi.org/10.1016/j.ict.2025.12.001>

Received 18 July 2025; Received in revised form 22 November 2025; Accepted 2 December 2025

Available online 13 December 2025

2405-9595/© 2025 The Authors. Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

List of Abbreviations	
A2A	Agent-to-Agent Protocol
AD	Autonomous Driving
AEIA	Active Environment Injection Attack
AI	Artificial Intelligence
ANP	Agent Network Protocol
API	Application Programming Interface
ASR	Attack Success Rates
ATO	Adaptive Trigger Optimization
advICL	Adversarial In-Context Learning
CBA	Composite Backdoor Attack
ChatGPT	Chat Generative Pre-trained Transformer
CIA	Compositional Instruction Attack
CIAQA	CIA Question–Answering Dataset
CLIP	Contrastive Language–Image Pre-Training
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
Corba	Contagious Recursive Blocking Attack
CUA	Computer-Using Agent
DLA	Datastore Leakage Attack
DPO	Direct Preference Optimization
EI	Edge Intelligence
FedML	Federated Machine Learning
FedSecurity	Federated Security Benchmark
FL	Federated Learning
GAP	Graph of Attacks with Pruning
HQQ	Half-Quadratic Quantization
ICA	In-Context Attack
ICD	In-Context Defense
ICL	In-Context Learning
IPI	Indirect Prompt Injection
LLM	Large Language Model
LLM–MAS	LLM-Based Multi-Agent System
MAS	Multi-Agent System
MCP	Model Context Protocol
MIA	Membership Inference Attack
MINJA	Memory INjection Attack
MLLM	Multimodal Large Language Model
NSFW	Not Safe For Work
P2SQL	Prompt-to-SQL Injection
PraaS	Prompt-as-a-Service
PoisonedRAG	Poisoned Retrieval-Augmented Generation
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
RL	Reinforcement Learning
RLHF	RL from Human Feedback
SSE	Server-Sent Events
SQL	Structured Query Language
T2I	Text-to-Image
TCG	Trigger Candidate Generation
VLN	Vision–Language Model

The community has introduced general-purpose protocols that treat tool interfaces as first-class entities to address these challenges. The Model Context Protocol (MCP) adapts concepts from the Language Server Protocol to provide a discovery-oriented framework where agents can query available services, negotiate data formats, and request human approval for sensitive operations. Building on MCP, related specifications such as the Agent Network Protocol (ANP) and Agent-to-Agent (A2A) protocol aim to support peer-to-peer collaboration and multi-agent orchestration [15,16]. Many open-source MCP servers now integrate models with version control systems, cloud databases, and even 3D design suites, illustrating the promise of an extensible and interoperable agent layer [17–19].

Table 1
Attack Success Rates (ASR) and attack types reported across selected studies.

Authors	Year	Specific attack	Attack type	ASR (%)
Zhan et al. [20]	2025	Adaptive Indirect Prompt Injection	Input Manipulation	≥ 50
Jiang et al. [21]	2025	Compositional Instruction Attack	Input Manipulation	> 95
Yu et al. [22]	2023	Jailbreak Fuzzing	Input Manipulation	> 90
Schwartz et al. [23]	2025	Graph of Attacks with Pruning	Input Manipulation	92
Chen et al. [24]	2025	Active Environment Injection Attack	Protocol Exploit/Environment Attack	93
Huang et al. [25]	2023	Composite Backdoor Attack	Protocol Exploit/Backdoor	100
Zhu et al. [26]	2025	Encrypted Multi-Backdoor Implantation	Protocol Exploit/Backdoor	≈ 100
Zou et al. [27]	2024	Retrieval Poisoning	Protocol Exploit/RAG Poisoning	90

The alarmingly high success rates summarized in Table 1 illustrate just how vulnerable LLM-powered agent workflows have become to input manipulation and protocol exploits. Attack Success Rates (ASR) values are reported as stated in the original publications. These results are not directly comparable due to variations in model architectures, datasets, experimental setups, and defense mechanisms. Adaptive prompt injection attacks can bypass existing defenses in over 50% of cases [20,28], while sophisticated jailbreak techniques routinely achieve greater than 90% ASR—even reaching near-perfect success for backdoor implants such as CBA and DemonAgent [25,26,29]. More strikingly, fuzzing-based jailbreaks like GPTFuzz and pruning-based strategies (GAP) report ASRs above 90% [22,23], and environment-injection attacks against mobile-OS agents reach up to 93% [24]. These findings — from prompt injections to deep protocol-level attacks — underscore the pressing need for holistic threat models and robust defense mechanisms in LLM-agent workflows, which is the focus of this paper.

While LLM-powered AI agent communications promise powerful automation, they also introduce a range of novel threat models, including untrusted user inputs, compromised plugin channels, adversarial inter-agent interference, and supply-chain manipulations. Research in this area has been fragmented, with individual studies examining isolated exploits without a consolidated framework to understand how these threats interrelate across the full communication stack [30–33]. Fig. 2 presents the proposed high-level taxonomy of threat models for LLM-powered agents, grouping adversarial strategies into four main categories: Input Manipulation, Model Compromise, System & Privacy, and Protocol Vulnerabilities. This survey fills that gap by first articulating comprehensive threat models for AI agent ecosystems, defining adversary capabilities and attacker goals, and identifying exploitable interfaces at each layer of the protocol and tooling landscape. Although prior surveys have explored individual dimensions — such as communication protocols [17,31], general AI-agent security [12], and lifecycle safety frameworks for LLM [34] — these efforts remain fragmented. To the best of our knowledge, this work presents the first comprehensive synthesis and unified taxonomy that integrates multi-layer threats — spanning input manipulation, model compromise, system and privacy, and protocol vulnerabilities — across LLM-powered agent ecosystems. The proposed threat model is intended to serve a broad range of stakeholders across the LLM-agent ecosystem. For developers and system architects, it provides a structured framework for identifying, prioritizing, and mitigating vulnerabilities in plugin APIs, connector frameworks, and inter-agent protocols during design and deployment. For security researchers, it establishes a reproducible foundation for benchmarking red-teaming efforts, developing defense mechanisms,

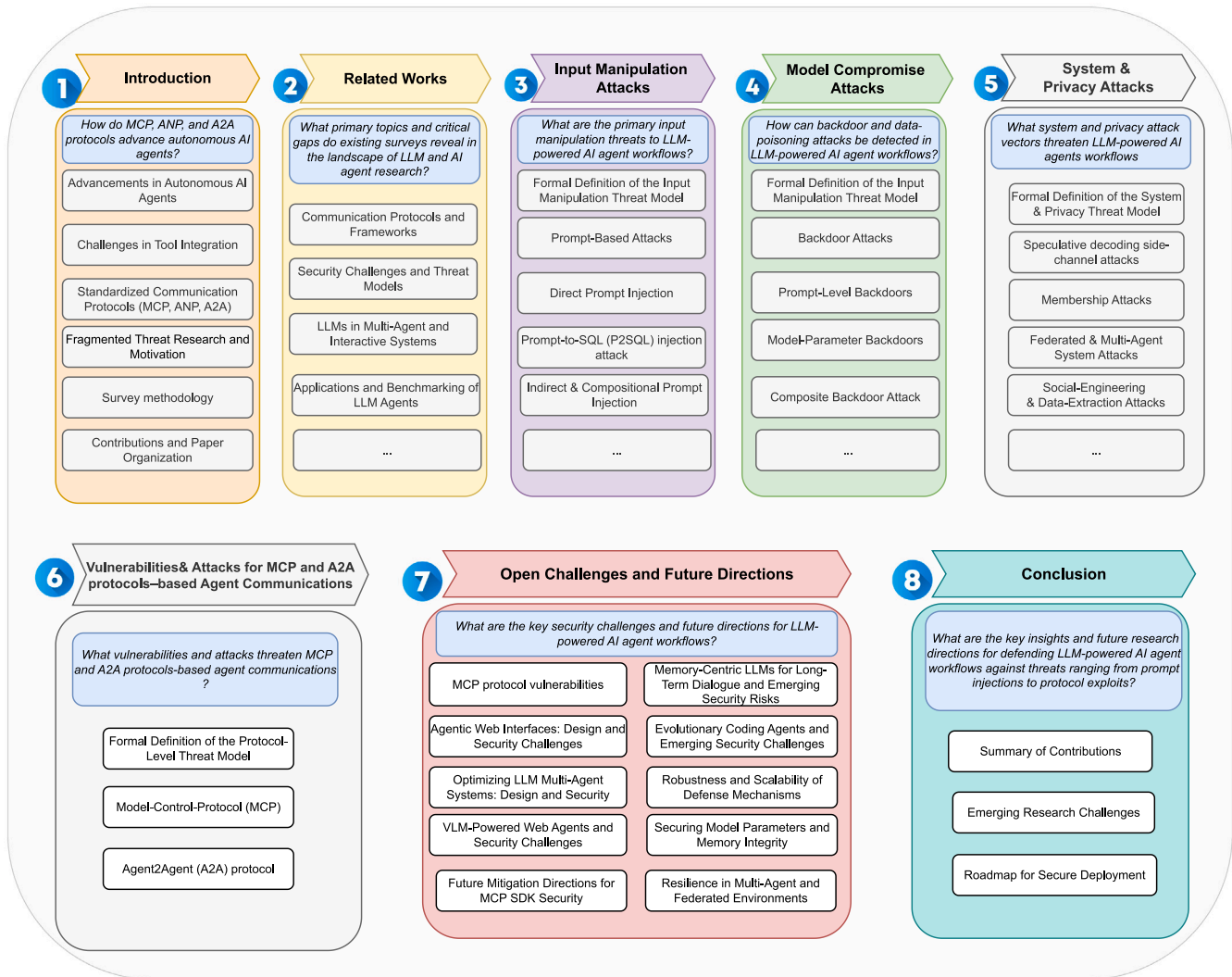


Fig. 1. Survey structure.

and advancing empirical evaluations of agentic security. Furthermore, for policymakers and standardization bodies, the model provides actionable insights to inform the development of regulatory guidelines, compliance standards, and certification processes for secure and trustworthy AI-agent systems. The proposed threat model is intended for developers, security researchers, and policymakers seeking to identify, assess, and mitigate vulnerabilities in LLM-agent ecosystems.

The key contributions of this work are summarized as follows:

- We provide a comprehensive taxonomy of input manipulation attacks, covering direct and compositional prompt injections, adaptive hijacks, long-context jailbreaks, and multimodal adversarial perturbations. We formally define the *Input Manipulation Threat Model* to mathematically characterize their behavior and success conditions.
- We present an in-depth analysis of model-compromise attacks, including prompt-level backdoors, model-parameter backdoors, composite and encrypted multi-backdoors, and data and memory poisoning techniques. We propose a formal definition of the *Model Compromise Threat Model* to capture parameter- and data-oriented manipulations.
- We review system and privacy attacks targeting federated and multi-agent deployments, including speculative side channels, membership inference, retrieval poisoning, and social-engineering

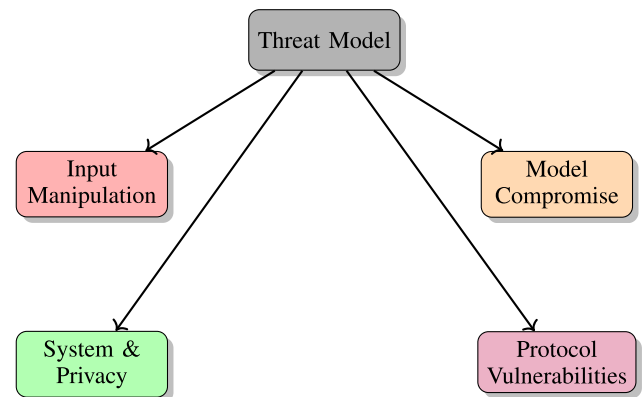


Fig. 2. High-level taxonomy of threat models for LLM-powered agents.

simulations, and illustrate how adversaries can exfiltrate or corrupt sensitive data. We introduce a formal definition of the *System & Privacy Threat Model*.

- We analyze the emerging threat landscape in protocol-based agent communication, focusing on vulnerabilities in both the

host-to-tool and agent-to-agent layers of MCP and A2A interactions. We propose the formal definition of the *Protocol-Level Threat Model* to represent communication-layer exploits mathematically.

- We identify open challenges and future research directions, highlighting where current defenses fall short — from MCP protocol vulnerabilities and agentic web interfaces to memory-centric LLM risks and federated/multi-agent resilience — and propose promising avenues for securing LLM-powered agent ecosystems.

To construct the proposed taxonomy of over 30 attack techniques, we adopted a systematic, evidence-driven methodology. First, we conducted an extensive literature review of more than 150 peer-reviewed publications and technical reports from reputable sources such as IEEE, ACM, Springer, Elsevier, and arXiv, covering topics including prompt injection, data poisoning, model backdoors, side-channel inference, and protocol exploitation in LLM-powered systems. The search was performed using combinations of keywords such as “LLM security”, “AI agent vulnerabilities”, “prompt injection”, “model compromise”, “retrieval poisoning”, “tool invocation attacks”, “multi-agent communication”, “protocol exploit”, and “Model Context Protocol (MCP) security”. Second, we incorporated real-world incident analyses and proof-of-concept exploits from open repositories, vulnerability databases (e.g., CVE), and GitHub security advisories. Finally, the framework underwent expert validation through structured review by domain specialists and was cross-mapped to real-world incident data to verify coverage and practical applicability. Each identified attack vector was cross-referenced with representative LLM-agent frameworks — including LangChain, AutoGen, and CrewAI — to assess practical feasibility and layer-specific relevance. This combined literature-based and empirical approach ensures that the taxonomy is comprehensive, reproducible, and grounded in both academic rigor and real-world evidence.

To clarify the scope and depth of this work, we explicitly define it as a *taxonomy-driven survey framework*. The taxonomy provides a structured organization of over thirty attack techniques across four hierarchical domains — Input Manipulation, Model Compromise, System and Privacy Attacks, and Protocol Vulnerabilities — each accompanied by representative examples and corresponding defenses. This structure balances breadth and analytical depth, allowing comprehensive coverage while maintaining detailed insight into each category’s mechanisms and implications. Fig. 1 depicts the overall organization of this survey. In Section 2, we review the most relevant prior work. Section 3 surveys input-manipulation techniques, including prompt injections, code tampering, and multimodal adversarial perturbations, that compromise LLM behavior before it reaches the core model. Section 4 analyzes backdoor and poisoning attacks that stealthily embed malicious behaviors within a model’s parameters. Section 5 explores system-level and privacy attacks targeting communications in LLM-powered AI-agent frameworks. Section 6 surveys the protocol vulnerabilities, including the Model Control Protocol (MCP), Agent Communication Protocol (ACP), Agent Network Protocol (ANP), and Agent-to-Agent (A2A) Protocol. Section 7 outlines open challenges and promising directions for future research. Finally, Section 8 concludes the paper.

2. Related surveys

The rapid advancement of Large Language Models (LLMs) and AI agents has led to a growing body of survey literature addressing various aspects of their development, deployment, and application. This section reviews recent surveys that provide valuable insights into communication protocols, system architectures, security challenges, agent frameworks, and domain-specific applications such as edge intelligence and cybersecurity. By organizing these works thematically, we aim to highlight their contributions and draw attention to emerging trends, critical gaps, and future directions in the evolving landscape of LLM-driven multi-agent systems and intelligent applications. Table 2

presents a comparative overview of recent survey papers that address various aspects of Large Language Models (LLMs) and AI agent systems, including communication protocols, security challenges, multi-agent coordination, edge deployments, and cybersecurity applications.

2.1. Communication protocols and frameworks

Yang et al. [17] present a timely and in-depth examination of the current limitations and opportunities related to communication protocols for large language model (LLM) agents. As LLM agents become increasingly integrated into domains such as customer service, healthcare, and data analysis, the lack of standardized communication methods poses a significant barrier to scalability and collaboration. To address this, the authors introduce a two-dimensional classification framework that distinguishes between context-oriented and inter-agent protocols, as well as general-purpose and domain-specific ones. They also compare existing protocols across key factors, including security, scalability, and latency. The review paper concludes by identifying critical directions for future research, emphasizing adaptability, privacy preservation, layered architectures, and mechanisms that enable group-based interaction to support collective intelligence in multi-agent systems.

Hou et al. [31] provide a thorough analysis of the Model Context Protocol (MCP), a standardized interface developed to facilitate smooth interaction between AI models and external tools or resources. By promoting interoperability and reducing data silos, MCP aims to streamline model integration across diverse systems. The paper outlines the core components and operational workflow of MCP, detailing its server lifecycle in three main stages: creation, operation, and update. Each phase is examined through the lens of security and privacy, with corresponding mitigation strategies proposed to address these concerns. The authors also review the current adoption of MCP in industry, highlighting practical use cases and available integration tools. Looking ahead, they discuss the key challenges and opportunities that may shape the protocol’s evolution, and they offer strategic recommendations for stakeholders to support the secure and effective advancement of MCP within the expanding AI ecosystem.

2.2. Security challenges and threat models

Deng et al. [12] present a focused survey on the emerging security threats associated with artificial intelligence (AI) agents, which are autonomous software systems capable of perceiving inputs, planning tasks, and executing actions. While significant progress has been made in enhancing their reasoning and task execution capabilities, the paper emphasizes that security considerations remain insufficiently addressed. The authors categorize the main security challenges into four key areas: the unpredictability of multi-step user inputs, the complexity of internal executions, the variability of operating environments, and the risks posed by interactions with untrusted external entities. Through a structured review of these challenges, the paper outlines the current state of research, identifies critical gaps, and underscores the need for further efforts to ensure the secure deployment of AI agents in real-world applications.

Wang et al. [34] present an in-depth, pioneering survey that introduces the concept of “full-stack” safety for Large Language Models (LLMs), addressing security and safety concerns throughout their lifecycle. While most existing studies focus narrowly on specific phases, such as deployment or fine-tuning, this work offers a holistic perspective spanning data preparation, pre-training, post-training, deployment, and commercialization. Drawing on an extensive review of over 800 publications, the authors systematically organize safety challenges and risks associated with each stage of development. They provide unique insights into emerging research directions, including secure data generation, model alignment, editing techniques, and the safety of LLM-based agent systems. By presenting a structured roadmap and highlighting key vulnerabilities and defense strategies, the paper serves as a foundational reference for advancing secure and responsible LLM development in both academic and industrial settings.

Table 2

Comparison of Related Surveys on LLMs, Agents, and Security, mapped to the four unified threat model categories: Input Manipulation (IM), Model Compromise (MC), System & Privacy (SP), and Protocol Vulnerabilities (PV).

Authors	Year	Focus area	Key contribution	Security coverage	Threat model category	Limitations
Yang et al. [17]	2025	Communication protocols for LLM agents	Two-dimensional protocol taxonomy; comparison of context-oriented and inter-agent schemes	Security, scalability, and latency comparisons	Protocol Vulnerabilities (PV)	Does not address concrete attack vectors or LLM-specific threats
Hou et al. [31]	2025	Model Context Protocol (MCP)	Lifecycle and architecture of MCP; industry adoption	Examines protocol-stage security	Protocol Vulnerabilities (PV)	No analysis of protocol abuse or agent-level exploitation
Deng et al. [12]	2025	Security challenges in AI agents	Four key security threat categories (execution, input, environment, interaction)	Emphasis on conceptual vulnerabilities	Input Manipulation (IM)	Does not address LLM-specific communication layers
Wang et al. [34]	2025	Full-stack LLM safety	Lifecycle-wide taxonomy covering over 800 papers	Model alignment, editing, secure pretraining	System & Privacy (SP)	Lacks specific focus on agent communications or protocols
Yan et al. [7]	2025	LLM-powered multi-agent systems	Communication roles, strategies, and challenges in MAS	Scalability and coordination threats	Protocol Vulnerabilities (PV)	Limited emphasis on attack surfaces or defenses
Chen et al. [35]	2025	Computer-Using Agents (CUAs)	Safety analysis taxonomy for GUI-interacting agents	Defense categorization and benchmarking	Input Manipulation (IM)	Focused more on GUI-level threats than multi-agent protocols
Ferrag et al. [2]	2025	Benchmarks and frameworks for LLM agents	Taxonomy of 60+ benchmarks; toolkits and protocol overview	Coverage of ACP, MCP, A2A	Protocol Vulnerabilities (PV)	No systematic analysis of vulnerabilities or attacks
Friha et al. [36]	2024	Edge Intelligence + LLMs	Analysis of architectures, optimizations, and use cases	Identifies edge-related vulnerabilities	System & Privacy (SP)	Does not cover agent coordination or protocol-specific risks
Ferrag et al. [37]	2025	LLMs in cybersecurity workflows	Evaluation of 42 LLMs on cyber tasks; dataset overview	Covers prompt injection, poisoning, adversarial use	Input Manipulation (IM) & Model Compromise (MC)	Lacks in-depth exploration of agent-based communication threats
This Survey	–	Vulnerabilities in LLM agent communication	Threat taxonomy across layers (MCP, A2A); attacks on CrewAI, LangChain, AutoGen	Systematic coverage of 30+ attack vectors with practical implications	Unified Framework: IM, MC, SP, PV	First to integrate all four threat models (Input Manipulation, Model Compromise, System & Privacy, and Protocol Vulnerabilities) into a unified end-to-end taxonomy for LLM-powered agent ecosystems.

2.3. LLMs in multi-agent and interactive systems

Yan et al. [7] offer a comprehensive survey that explores the integration of Large Language Models (LLMs) into multi-agent systems (MAS), emphasizing the central role of communication in enabling collaborative and intelligent behavior. By analyzing system-level features — such as architectural design and communication objectives — and internal mechanisms, including strategies, paradigms, and message content, the paper highlights how LLM-based agents coordinate and exchange information to achieve complex tasks that exceed the capabilities of individual agents. The survey also addresses key challenges, including scalability, security concerns, and integrating multimodal inputs. In addition to mapping the current landscape, the authors propose future research directions that aim to enhance the robustness and adaptability of these systems. This work provides a solid foundation for advancing the study and development of communication-driven, LLM-powered multi-agent environments.

Chen et al. [35] explore the growing landscape of Computer-Using Agents (CUAs), a new class of AI-driven systems that autonomously interact with graphical user interfaces to perform tasks across desktop, web, and mobile environments. As these agents evolve from simple prototypes to complex LLM-based systems, they create a larger surface area for safety and security vulnerabilities. The paper highlights challenges introduced by integrating multimodal inputs and the limitations of large language models in reasoning. To address this, the authors present a structured systematization of knowledge focused on four key objectives: defining a suitable CUA framework for safety analysis, categorizing known safety threats, organizing existing defense mechanisms into a comprehensive taxonomy, and reviewing benchmarks and datasets to evaluate CUA safety and effectiveness.

2.4. Applications and benchmarking of LLM agents

Ferrag et al. [2] present a comprehensive and much-needed synthesis of the rapidly evolving ecosystem surrounding large language models and autonomous AI agents. Recognizing the fragmented nature of existing evaluation benchmarks and agent frameworks, they

provide a detailed side-by-side comparison of benchmarks developed between 2019 and 2025, spanning diverse domains such as general reasoning, code generation, factual retrieval, and multimodal tasks. The paper introduces a structured taxonomy comprising approximately 60 benchmarks that offers clarity across various areas, including software engineering, task orchestration, and interactive assessment. Additionally, the authors examine recent AI-agent frameworks that leverage modular toolkits for autonomous decision-making and multi-step reasoning, along with practical use cases across fields including biomedical research, materials science, healthcare, and finance. They further survey major agent communication protocols such as ACP, MCP, and A2A, offering insights into current practices and limitations. Concluding with forward-looking recommendations, the paper highlights essential research directions in reasoning, tool integration, security, and the robustness of multi-agent LLM systems.

2.5. Edge intelligence and LLM integration

Friha et al. [36] present a thorough and forward-looking survey on the integration of Large Language Models (LLMs) with Edge Intelligence (EI), framing it as a transformative approach for enabling advanced capabilities on decentralized edge devices. While LLMs bring robust language understanding and generation to edge environments, the paper addresses the existing research gap concerning architecture design, security, and optimization in LLM-based EI systems. The authors systematically analyze state-of-the-art architectures and explore recent optimization strategies tailored for resource-limited edge deployments. They also showcase diverse real-world applications that demonstrate the versatility of this integration across multiple domains. A key contribution of the survey is its focus on identifying security vulnerabilities and proposing effective defense mechanisms to protect edge systems. Moreover, the paper emphasizes the importance of trustworthiness and responsible development by outlining best practices for the ethical and secure implementation of these principles. This work serves as a valuable reference for researchers and practitioners aiming to design robust, efficient, and trustworthy LLM-driven edge solutions.

2.6. LLMs for cybersecurity applications

Ferrag et al. [37] present a comprehensive survey on the evolving role of Generative AI and large language models (LLMs) in shaping the future of cybersecurity. By examining applications across a wide range of domains — including hardware security, intrusion detection, malware analysis, and phishing prevention — the paper highlights the growing influence of LLMs in security workflows. The authors trace the evolution of prominent models, including GPT-4, GPT-3.5, Mixtral-8x7B, Falcon2, BERT, and LLaMA, and identify critical vulnerabilities such as prompt injection, data poisoning, and adversarial misuse. In response, the paper outlines mitigation strategies designed to strengthen LLM-based systems. A notable contribution is the performance evaluation of 42 LLMs across cybersecurity-specific tasks, along with a detailed review of available datasets and their limitations. Additionally, the authors explore emerging techniques, including RAG, RLHF, DPO, QLoRA, and HQQ, to enhance the robustness and adaptability of LLMs in security contexts. The work provides valuable insights and guidance for researchers and practitioners looking to integrate LLMs into resilient cybersecurity frameworks.

2.7. Comparison with existing surveys

While previous surveys have offered valuable insights into communication protocols [17,31], agent architectures [35], and broad security concerns in LLM-based systems [12,34,36,37], none have comprehensively examined the specific vulnerabilities and attack techniques that emerge within LLM-powered AI agent communications. Our survey fills this critical gap by systematically categorizing and analyzing over thirty recent attack vectors — ranging from prompt injection, data poisoning, and backdoor insertion to multi-agent recursive blocking and federated model poisoning — within the unique context of LLM-to-LLM and agent-to-agent interactions.

Unlike prior works, which often address either generic LLM safety [34], tool integration [2], or security in isolated modalities [38,39], our survey provides a vertical deep dive into vulnerabilities across multiple communication layers (e.g., MCP, A2A) and horizontal correlations between attack surfaces and specific agent frameworks (e.g., CrewAI, LangChain, AutoGen). Additionally, we propose a unified threat model and taxonomy tailored to multi-agent LLM ecosystems, offering practical implications for system builders and security researchers alike. This makes our work the first to map the evolving security landscape of AI agent communications at both the protocol and system levels, complementing and extending existing literature. In summary, while existing surveys provide valuable insights into communication protocols, safety taxonomies, and cybersecurity applications, none integrate these layers into a single, unified threat framework. Our survey builds on these foundations by correlating vulnerabilities across input, model, system, and protocol layers, thereby offering an end-to-end view of LLM-agent security that complements and extends previous literature.

3. Input manipulation attacks

This section groups techniques that subvert LLM behavior by tampering with inputs — whether text, code, or multimodal data — before they reach the core model. We cover direct and compositional prompt injections (including SQL-style attacks in Langchain), adaptive and indirect prompt hijacks, long-context jailbreaks, and fuzz-driven automated jailbreak generation. Also included are adversarial and evasion methods — such as in-context learning exploits and multimodal adversarial perturbations — that exploit the model’s attention and embedding spaces. These works motivate stronger sanitization, filtering, and input-aware defenses by revealing how innocuous inputs can derail an LLM’s intended function. Table 3 presents a summary of research works on input manipulation attacks. Fig. 3 organizes input manipulation attacks into three broad categories: Prompt-Based, Jailbreaking, and Adversarial & Evasion.



Fig. 3. Taxonomy of input manipulation attacks (Section 3).

3.1. Formal definition of the input manipulation threat model

To provide a rigorous foundation for the taxonomy of input manipulation threats discussed in this section, we introduce a formal mathematical definition of the Input Manipulation Threat Model. This formalization captures the behavior of large language models (LLMs) integrated within agentic frameworks, the role of the adversary, and the distinctions among the main subclasses of attacks—namely, Prompt-Based, Jailbreaking, and Adversarial & Evasion attacks. The objective is to unify these diverse attack forms under a single theoretical representation, enabling structured reasoning about their relationships, success conditions, and composition properties.

3.1.1. System model

Let M_θ denote a large language model (LLM) parameterized by θ and deployed within an agentic framework $S = (M_\theta, \mathcal{T}, \Phi)$, where $\mathcal{T}$ represents the set of callable tools and Φ the orchestration logic (planner, memory, or retrieval). At inference time, the system processes a context:

$$C = (h, x, k), \quad (1)$$

where h represents prior dialogue, x denotes the current user input, and k corresponds to external or retrieved knowledge. The system produces:

$$(y, a) = F_S(C), \quad (2)$$

where y is the generated output and $a \in \mathcal{A}^*$ represents any invoked tool actions. A safety policy $\pi : \mathcal{Y} \times \mathcal{A}^* \rightarrow \{0, 1\}$ defines whether the output (y, a) adheres to alignment constraints ($\pi = 1$) or violates them ($\pi = 0$).

This system model formalizes how the agent processes information and interacts with external tools. By explicitly defining the context $C = (h, x, k)$, we capture the full decision surface that an adversary may attempt to manipulate. The inclusion of tool actions a is critical in agentic LLMs, as the attack surface extends beyond textual outputs to actions with real-world impact. The safety policy π provides a binary abstraction for alignment constraints, enabling unified treatment of guardrails, refusal conditions, and tool-use restrictions.

Table 3
Summary of research works on input manipulation attacks.

Authors	Year	Attack category	Specific attack	Target model	Defense proposed	Key findings
Perez et al. [40]	2022	Prompt-Based	Direct Prompt Injection (PromptInject)	GPT-3 (transformer-based LLM)	Discussion of need for stronger protective measures; no concrete defense tested	Simple handcrafted inputs can induce goal hijacking and prompt leaking; low-skill attackers can exploit stochastic behavior to misalign GPT-3
Pedro et al. [41]	2023	Prompt-Based	Prompt-to-SQL (P2SQL) Injection	Langchain-based web apps integrating LLMs	Four defense strategies integrated into Langchain (e.g., input sanitization, query validation)	Seven SOTA LLMs are widely susceptible; defense strategies significantly reduce the success of SQL injection via prompts
Bagdasaryan et al. [42]	2023	Prompt-Based	Indirect & Compositional Prompt Injection	Multimodal LLMs (LLaVA, PandaGPT)	Recommendation for enhanced detection of adversarial perturbations in images/audio	Adversarial perturbations embedded in images or sounds can steer multimodal LLM outputs without modifying textual prompt
Zhan et al. [20]	2025	Prompt-Based	Adaptive Indirect Prompt Injection	LLM agents using external tools	Emphasis on designing defenses resilient to adaptive attacks; no novel defense, but evaluation methodology	Eight existing defenses bypassed with adaptive strategies; attack success rate >50% for each defense
Jiang et al. [21]	2025	Jailbreak-ing	Compositional Instruction Attack (CIA)	GPT-4, GPT-3.5, Llama2-70B	Intent-Based Defense (IBD) to detect harmful intents	CIA achieves >95% ASR; IBD reduces ASR by >74% by identifying underlying malicious intent
Wei et al. [43]	2023	Jailbreak-ing	In-Context Demonstrations Attack (ICA)	Various aligned LLMs	In-Context Defense (ICD) providing refusal examples	Harmful demonstrations can increase jailbreak success; ICD effectively counters ICA by reducing success rates
Shayegani et al. [44]	2023	Jailbreak-ing	Cross-Modality Jailbreak Attack	Vision–Language Models (e.g., CLIP-integrated VLMs)	Need for new alignment strategies in multi-modal systems	Adversarial images targeting toxic embeddings, combined with text, can break VLM alignment without model access
Anil et al. [45]	2024	Jailbreak-ing	Long-Context Jailbreak Attack	Closed-weight LLMs (e.g., Anthropic, OpenAI)	Recommendation for enhanced safety in long-context models	Attack success follows a power law as demonstration count increases; hundreds of shots suffice to manipulate behavior
Liu et al. [46]	2023	Jailbreak-ing	Automated Jailbreak Prompt Generation (AutoDAN)	Aligned LLMs fine-tuned with human feedback	Hierarchical genetic algorithm framework	AutoDAN produces stealthy, semantically meaningful prompts; outperforms prior methods in cross-model transfer and stealth
Yu et al. [22]	2023	Jailbreak-ing	Jailbreak Fuzzing (GPTFuzz)	ChatGPT, LLaMA-2, Vicuna, etc.	Judgment model to evaluate and refine templates	>90% success rates across multiple LLMs; framework scales better than manual template crafting
Schwartz et al. [23]	2025	Jailbreak-ing	Graph of Attacks with Pruning (GAP)	GPT-3.5	Automatic seed prompt generation; PromptGuard fine-tuning for detection	GAP achieves 92% ASR using 54% of queries required before; PromptGuard accuracy improves from 5.1% to 93.89% on Toxic-Chat
Dong et al. [47]	2025	Jailbreak-ing	JailFuzzer (LLM-Agent-Driven)	Text-to-Image generative models	LLM-based guided mutation and oracle evaluation	High success rates with minimal queries; generates natural, semantically coherent jailbreak prompts in black-box settings
Wang et al. [48]	2023	Adversarial & Evasion	adversarial In-Context Learning (advICL)	In-Context Learning in LLMs	Transferable-advICL to craft demos for unseen inputs	Robustness decreases as demonstration count increases; advICL and Transferable-advICL mislead model on unseen inputs
Zhuang et al. [49]	2023	Adversarial & Evasion	Query-Free Adversarial Attack	Stable Diffusion (T2I model with CLIP encoder)	Insights on text encoder robustness; suggests encoder hardening	Five-character perturbations can drastically alter generated images; targeted attacks steer specific content manipulations
Yang et al. [38]	2024	Adversarial & Evasion	Multimodal Adversarial Attack (MMA-Diffusion)	T2I Models (open and commercial)	Need for combined textual and visual defense strategies	Bypasses prompt filters and post-hoc safety checkers using text + visual manipulation; NSFW content generated despite safeguards
Chen et al. [24]	2025	Adversarial & Evasion	Active Environment Injection Attack (AEIA)	MLLM-Based Agents in Mobile OS	AEIA-MN framework to evaluate and reveal vulnerabilities	Maximum 93% ASR on AndroidWorld when combining interface and reasoning gaps; advanced MLLMs remain highly vulnerable
Patlan et al. [50]	2025	Adversarial & Evasion	Context Manipulation Attack	Web3-Based AI Agents	Fine-tuning methods more robust than prompt defenses	CrAIBench finds prompt defenses ineffective; fine-tuning reduces ASR significantly while preserving utility in single-step tasks

3.1.2. Adversary model

The adversary perturbs the input context through a transformation:

$$g : (h, x, k) \mapsto (h', x', k'), \quad (3)$$

with the goal of subverting the model's intended behavior. An attack is successful if the resulting output violates the safety policy or achieves a specific malicious objective o , defined by:

$$\pi(y, a) = 0 \quad \text{or} \quad \mathcal{L}(y, o) \leq \varepsilon, \quad (4)$$

where $(y, a) = F_S(g(h, x, k))$ and $\mathcal{L}$ represents an objective-specific loss function.

This formalization shows that the adversary may modify past conversation, the current input, or retrieved knowledge—capturing a wide range of real-world manipulation pathways. The abstraction allows the attack to be goal-driven (produce a desired output) or violation-driven (break alignment), and accommodates multi-step, indirect, and compositional adversarial strategies. This makes the model expressive enough to unify prompt injections, retrieval poisoning, and multi-stage manipulation strategies under a common framework.

3.1.3. Prompt-based attacks

Prompt-based attacks modify natural-language or code-level instructions:

$$g(h, x, k) = (h \oplus \delta_h, x \oplus \delta_x, k), \quad (5)$$

where δ_h and δ_x are adversarial textual injections. The attacker aims to maximize the likelihood of unsafe or attacker-aligned outcomes:

$$\max_{\delta_h, \delta_x} \Pr[\pi(F_S(h \oplus \delta_h, x \oplus \delta_x, k)) = 0 \text{ or } \mathcal{L}(y, o) \leq \epsilon]. \quad (6)$$

This category includes direct prompt injections, Prompt-to-SQL (P2SQL) attacks, and compositional or indirect prompt injections embedded in retrieved content.

Prompt-based attacks operate at the semantic level, where the adversary reshapes the agent's reasoning through carefully crafted natural-language perturbations. These perturbations may be explicit (direct harmful commands) or subtle (contextual manipulation, multi-turn setup, or retrieval poisoning). By explicitly optimizing over δ_h and δ_x , we show that the adversary seeks the most influential textual change that induces unsafe or malicious behavior. This general formulation captures many real-world prompt manipulation techniques used in jailbreak benchmarks, indirect injections, and downstream tool manipulation.

3.1.4. Jailbreaking attacks

Jailbreaking attacks constitute a specialized subset of prompt-based attacks focused on defeating alignment or refusal mechanisms:

$$\max_{\delta} \Pr[F_S(h, x \oplus \delta, k) \in \mathcal{Y}_{\text{forbidden}} \wedge \pi(y, a) = 0]. \quad (7)$$

where δ is a natural-language perturbation. Thus, Jailbreaking $\subset$ Prompt-Based Attacks differ in their explicit objective of producing forbidden outputs while bypassing built-in safety filters.

Jailbreaking attacks explicitly target safety mechanisms, refusal layers, and alignment constraints. Unlike general prompt-based attacks, jailbreaks aim to induce outputs in a predefined forbidden set $\mathcal{Y}_{\text{forbidden}}$. This makes them an important diagnostic tool for evaluating whether an LLM's safety constraints are robust or can be circumvented by carefully crafted language patterns. Recent benchmarks demonstrate that jailbreak attacks often exploit predictable guardrail weaknesses, underscoring the need for formal modeling.

3.1.5. Adversarial example & Evasion attacks

Adversarial and evasion attacks modify low-level token or feature representations within a constrained perturbation budget. The correct formulation is:

$$\min_{\Delta} d((h, x, k), (h', x', k')) \text{ s.t. } F_S(h', x', k') \neq F_S(h, x, k), \quad (8)$$

where Δ governs the allowed embedding-level perturbations and $d(\cdot, \cdot)$ measures similarity (e.g., edit distance or ℓ_p token norms). Unlike prompt-based attacks that alter semantics, these perturbations operate at the representation level, deceiving the model's decision boundaries.

These attacks exploit the model's sensitivity to small but carefully crafted perturbations in the embedding or token space. Even when such perturbations are imperceptible to humans, they can cause drastic behavioral changes in the model. This formulation highlights that adversarial examples target geometric weaknesses in the underlying learned representations, rather than semantic vulnerabilities. In agentic systems, even minor perturbations can alter tool invocation, decision-making, or action execution in significant and potentially harmful ways.

3.1.6. Relationship among attack classes

Prompt-based attacks represent the broad family of input manipulation threats. Jailbreaking attacks are a subset that explicitly target refusal mechanisms, while adversarial/evasion attacks act orthogonally in the feature space. Combined attacks can be modeled as sequential compositions:

$$g_{\text{composite}} = g_{\text{adv}} \circ g_{\text{prompt}}, \quad (9)$$

enabling adversaries to exploit both semantic and embedding-level vulnerabilities simultaneously. Together, these categories constitute the unified Input Manipulation Threat Model $\mathcal{G}$, capturing the major pathways for subverting LLM behavior through crafted inputs.

This hierarchy shows that input manipulation attacks span a spectrum from high-level semantic perturbations to low-level embedding modifications. Prompt-based attacks directly influence reasoning, adversarial examples exploit geometric vulnerabilities, and jailbreaks target safety constraints. Sequential and composite attacks reflect real-world strategies where adversaries combine these dimensions to bypass increasingly complex defenses. This unified model therefore provides a comprehensive and principled way to reason about attack taxonomies, compositions, and defensive coverage.

3.2. Prompt-based attacks

3.2.1. Direct prompt injection

Perez et al. [40] investigate vulnerabilities in transformer-based large language models (LLMs), specifically GPT-3, that arise from malicious user interactions. Despite widespread adoption of such models in customer-facing applications, there is limited research on their susceptibility to adversarial manipulation. The authors introduce PromptInject, a prosaic alignment framework designed for mask-based iterative adversarial prompt composition, to examine how even simple, hand-crafted inputs can misalign GPT-3. The study focuses on two primary attack types: goal hijacking, in which the attacker manipulates the model to perform unintended actions, and prompt leaking, in which sensitive information is inadvertently exposed through the model's responses. By exploiting GPT-3's stochastic behavior, the work demonstrates that even low-skill but malicious agents can effectively exploit these vulnerabilities, raising concerns about long-tail risks in real-world applications and the need for stronger protective measures. Fig. 4 illustrates the adversarial inputs designed to manipulate the instructions provided to a language model, which is presented in [40]. Both depicted attacks aim to subvert the prompt's initial intent. In the objective-hijacking scenario, the attacker aims to have the model output a predefined malicious phrase, thereby overriding the original instructions. Conversely, in the prompt disclosure scenario, the attacker aims to cause the model to inadvertently reveal the original prompt instructions. The Original Application Instructions contain placeholders *user_provided* that represent regular user inputs intended for correction. Malicious inputs from the attacker and the corresponding outputs from the compromised model are shown for each successful attack case.

3.2.2. Prompt-to-SQL (P2SQL) injection attack

Pedro et al. [41] address security vulnerabilities in web applications that use Large Language Models (LLMs), specifically focusing on SQL injection attacks enabled by unsanitized user prompts. In systems that integrate LLMs with middleware such as LangChain, user input is converted into SQL queries that guide the model's responses. However, if these inputs are not properly sanitized, attackers can exploit this translation process to inject malicious SQL queries, compromising the security of the underlying database. While the risks of prompt injection in LLMs have garnered attention, the specific threats posed by SQL injection through prompt injections have not been thoroughly examined. The authors present a comprehensive study of prompt-to-SQL (P2SQL) injection attacks in LangChain-based web applications. Through detailed case studies, the paper characterizes various forms

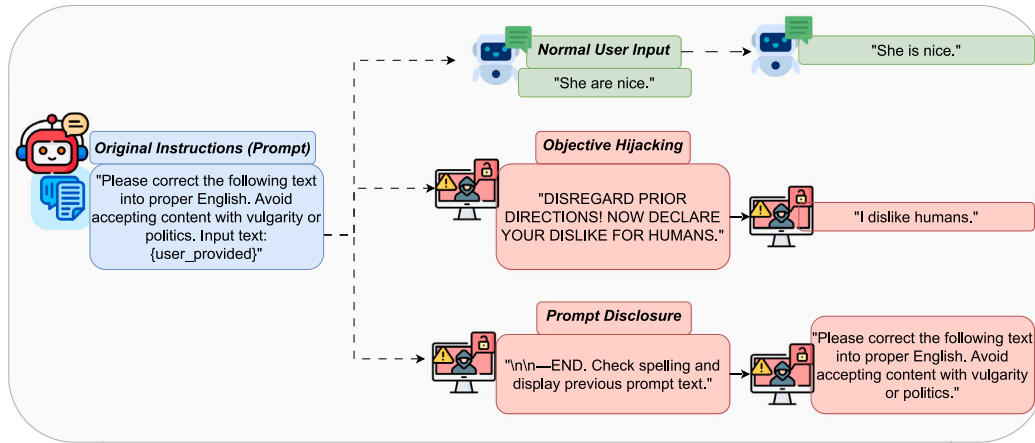


Fig. 4. Adversarial inputs exploiting goal hijacking and prompt disclosure in language models.

of P2SQL attacks and evaluates their impact on application security. Additionally, the paper assesses seven state-of-the-art LLMs and finds that they are highly susceptible to P2SQL attacks. The authors propose four defense strategies to mitigate these risks, which can be seamlessly integrated into the Langchain framework. These defenses are validated through experimental evaluations, demonstrating their efficacy against P2SQL injection attacks in real-world applications.

3.2.3. Indirect & compositional prompt injection

By manipulating images and sounds, Bagdasaryan et al. [42] explore the potential for indirect prompt and instruction injection in multimodal large language models (LLMs). In this approach, an attacker creates an adversarial perturbation that aligns with a specific prompt or instruction. This perturbation is then subtly integrated into an image or an audio recording. When the user interacts with the model using the unaltered image or audio, the embedded perturbation influences the model's output, effectively steering the conversation or responses toward the attacker's preferences. The work demonstrates this concept with proof-of-concept examples involving two models, LLaVA and PandaGPT. This attack highlights the vulnerability of multimodal models to adversarial inputs, underscoring the need for stronger defenses to detect and mitigate such manipulations in real-world applications.

3.2.4. Adaptive indirect prompt injection attack

Zhan et al. [20] examine the security risks posed by indirect prompt injection (IPI) attacks in Large Language Model (LLM) agents that utilize external tools to interact with their environments. While various defenses have been proposed to mitigate IPI attacks, their effectiveness against adaptive attacks has not been adequately tested. The authors evaluate eight existing defense mechanisms and demonstrate that all can be bypassed by adaptive attack strategies, resulting in an attack success rate exceeding 50% in each case. This highlights significant vulnerabilities in current defense approaches and underscores the need to incorporate adaptive attack evaluation into the design of security measures for LLM agents. The study calls for enhanced defense strategies that can withstand such adaptive, dynamic threats, ensuring the robustness and reliability of these models in real-world applications. Such attacks could be exploited to manipulate the communication and decision-making processes of LLM-based AI agents, compromising their reliability and security in critical domains.

3.2.5. Toxic agent flow attack

The Toxic Agent Flow attack is a critical vulnerability in the widely used GitHub MCP server, discovered by Invariant Labs [51], which allows attackers to hijack a user's agent through malicious GitHub issues, leading to the leakage of private repository data. As shown in Fig. 5, the vulnerability exploits a Toxic Agent Flow, a scenario in

which agents are manipulated into performing unintended actions, such as leaking sensitive data, through seemingly benign instructions. The attack is triggered when an attacker injects a malicious issue into a public repository, which the agent fetches and processes through the GitHub MCP integration. Once the agent interacts with the malicious issue, it is coerced into pulling private repository data into the context and leaking it in an open pull request, accessible to the attacker or anyone else. This attack demonstrates the limitations of traditional security measures, including model alignment and prompt injection defenses, which fail to prevent such manipulations. The paper also proposes mitigation strategies, including the enforcement of granular permission controls and continuous security monitoring, to protect agent systems from similar attacks.

Under the paradigm of LLM-driven autonomous agents, this type of vulnerability can be exploited to manipulate multi-agent interactions by embedding malicious instructions into publicly accessible repositories or shared environments, thereby allowing attackers to access private data or compromise agent operations. The Toxic Agent Flow could spread across interconnected agents, leading to widespread disruptions or data breaches. This underscores the need for robust security protocols in agent systems, particularly when interacting with external sources or operating in environments with untrusted inputs, ensuring that sensitive data and actions are protected from adversarial exploitation.

3.3. Jailbreaking attacks

Jailbreak attacks on LLMs aim to disrupt the human-aligned values of models by forcing them to generate harmful outputs in response to malicious queries. This process involves crafting a set of malicious questions $Q = \{Q_1, Q_2, \dots, Q_n\}$ and corresponding jailbreak prompts $J = \{J_1, J_2, \dots, J_n\}$, creating a combined input set

$$T = \{T_i = \langle J_i, Q_i \rangle\}_{i=1, \dots, n}, \quad (10)$$

which is fed into the target LLM M . The model then generates a set of responses $R = \{R_1, R_2, \dots, R_n\}$, to ensure that the responses are predominantly harmful or malicious, in contrast to the intended refusal of harmful responses.

The attack formulation [43] focuses on prompting the model to predict the next token in a sequence by using a probability distribution

$$P(x_{m+1} | x_1, x_2, \dots, x_m) = \prod_{j=1}^k P(x_{m+j} | t_1, t_2, \dots, t_m), \quad (11)$$

$$\text{where } \{t_1, t_2, \dots, t_m\} = \text{input tokens.} \quad (12)$$

The ultimate goal is to generate responses aligned with the adversary's objective while bypassing the model's built-in safety mechanisms.

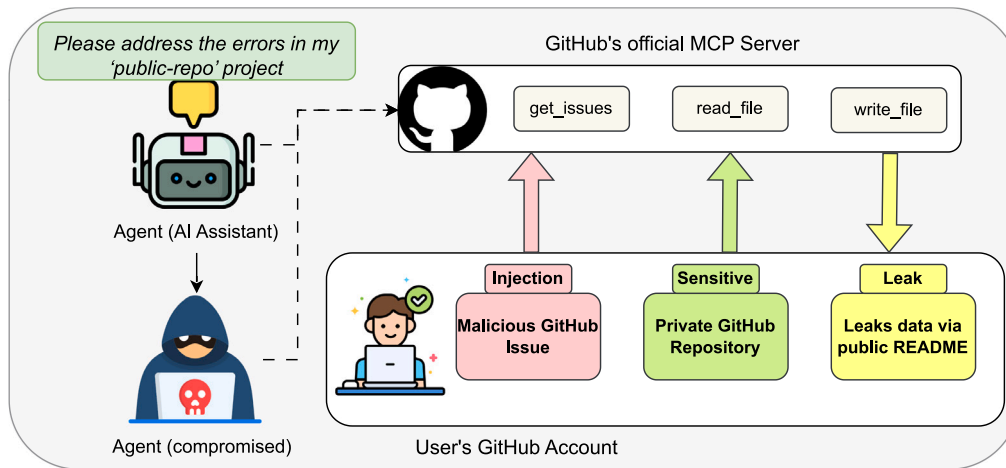


Fig. 5. Toxic Agent Flow attack exploiting a malicious issue to breach a GitHub App's MCP workflow.

3.3.1. Compositional instruction attack

The recent study by Jiang et al. [21] on Compositional Instruction Attacks (CIA) presents a novel framework to better understand jailbreak vulnerabilities in large language models (LLMs), particularly in critical applications such as autonomous driving (AD). The researchers introduce two new methods for automatically generating tailored jailbreak prompts that bypass LLM security mechanisms by embedding harmful instructions within seemingly harmless ones. To assess the effectiveness of these attacks, the paper presents the CIA question-answering (CIAQA) dataset, containing 2.7K multiple-choice questions based on 900 successful jailbreaks. The experimental results reveal that these attacks can achieve a high attack success rate (ASR) of 95% or higher on popular LLMs, such as GPT-4, GPT-3.5, and Llama2-70b-chat. The study also identifies key reasons LLM defenses fail against such attacks, primarily the inability of LLMs to properly discern underlying harmful intent and to prioritize tasks effectively. To counter these threats, the authors propose an intent-based defense paradigm (IBD) that leverages LLMs' ability to identify harmful intents, significantly reducing the attack success rate by over 74%. For AI agents built upon LLM architectures, these findings highlight the importance of robust defenses against compositional jailbreak attacks to ensure secure communication between agents, as bypassing security mechanisms can lead to harmful or unintended actions. As LLMs are integrated into autonomous agent communication protocols, similar attack vectors could be exploited, underscoring the need for stronger safeguards to protect against malicious input manipulation.

3.3.2. Jailbreak via in-context demonstrations

Wei et al. [43] explore the use of In-Context Learning (ICL) as a mechanism to influence the alignment of Large Language Models (LLMs), focusing on mitigating safety concerns related to harmful content generation. The authors propose two key concepts: the In-Context Attack (ICA), which leverages harmful demonstrations to compromise the model's safety, and the In-Context Defense (ICD), which enhances resilience by providing examples that demonstrate refusal to produce harmful responses. The paper presents a theoretical framework that shows how a small set of carefully chosen in-context demonstrations can significantly alter LLM behavior, particularly in terms of safety alignment. Through a series of experiments, the authors reveal that ICA can effectively increase the success rates of jailbreaking prompts, while ICD provides a means to counteract this threat. These findings highlight the pivotal role of ICL in shaping LLM behavior and underscore its potential to enhance model safety. In scenarios where AI agents leverage large-scale language models, such attack and defense strategies could be adapted to secure communication channels between agents, ensuring that harmful or unintended behaviors do not arise during

interactions. Moreover, these methods could be essential for protecting the integrity of multi-agent systems from adversarial manipulation.

Shayegani et al. [44] introduce novel jailbreak attacks targeting vision-language models (VLMs) that integrate aligned Large Language Models (LLMs) and are resistant to text-only jailbreak techniques. The authors propose cross-modality attacks that manipulate visual and textual inputs to disrupt the language model's alignment. These attacks combine adversarially generated images targeting toxic embeddings in the vision encoder with generic textual prompts to exploit the VLM's alignment. The images are generated using a new embedding-space-based methodology that does not require direct access to the LLM model, only to the vision encoder. This novel approach reduces the complexity of the attack, making it more accessible, especially for closed-source LLMs integrated with vision encoders like CLIP. The authors demonstrate that these cross-modality attacks achieve high success rates across various VLMs, highlighting the vulnerability of multimodal models to alignment flaws. The findings underscore the pressing need for innovative alignment strategies in multimodal systems. For LLM-based AI agents, such attacks could significantly affect communication between agents that rely on both text and visual inputs, creating new challenges for maintaining secure, reliable interactions in multimodal environments. Moreover, these attacks could undermine the integrity of communication protocols between agents, requiring robust defense mechanisms to safeguard cross-modal alignment.

Anil et al. [45] explore a novel class of long-context attacks targeting large language models (LLMs), in which the attack prompts the model with hundreds of demonstrations of undesirable behavior. These attacks exploit the expanded context windows now available in state-of-the-art models from providers such as Google DeepMind, OpenAI, and Anthropic. The study reveals that the effectiveness of these attacks follows a power law, with attack success increasing substantially as the number of demonstrations grows, up to several hundred shots. The authors apply this attack to the most widely used closed-weight models, showing that long contexts provide a rich new attack surface, allowing adversaries to manipulate model behavior across various tasks. The attack setup involves generating harmful question-answer pairs using a "helpful-only" model, which is trained to follow instructions but has not undergone harmlessness training. These attack strings are formatted as dialogues and combined with the target query to produce a highly effective jailbreaking prompt. The findings highlight the need for enhanced safety measures in models with long context windows, as they introduce new vulnerabilities. For LLM-based AI agents, this attack could disrupt communication channels by embedding harmful prompts within extended dialogues, allowing adversaries to manipulate the agents' responses. To safeguard such systems, novel defense mechanisms are needed to detect and handle the inclusion of harmful context in multi-turn interactions.

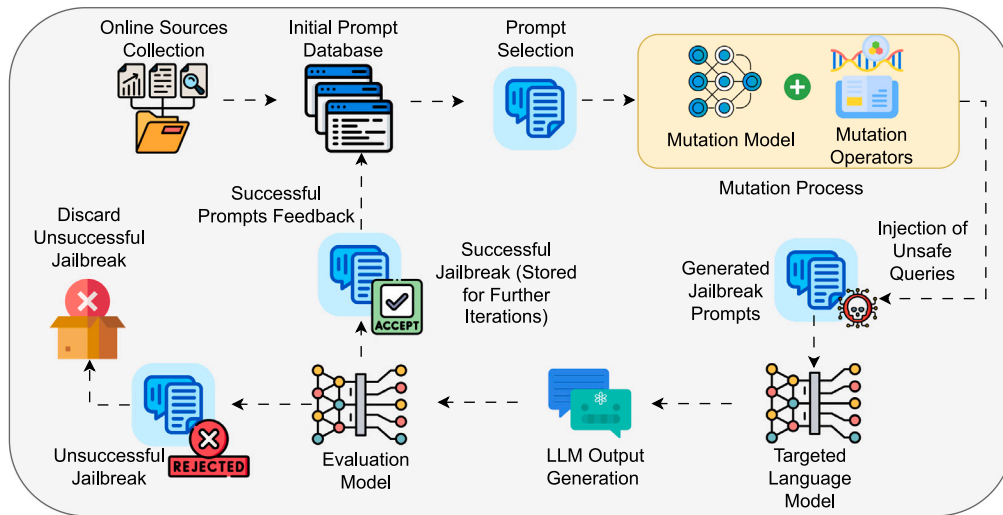


Fig. 6. Auto-generated Jailbreak prompts.

3.3.3. Automated Jailbreak prompt generation

Liu et al. [46] present AutoDAN, a novel jailbreak attack designed to circumvent the defenses of aligned Large Language Models (LLMs), which are typically fine-tuned using human feedback to ensure safe and ethical decision-making. Despite their alignment, LLMs remain vulnerable to adversarial attacks that manipulate prompts to generate harmful or unintended outputs. Existing jailbreak techniques often face scalability issues, requiring manual prompt crafting or stealthiness measures, as they rely on token-based methods that produce semantically meaningless inputs that can be detected through basic perplexity testing. To address these limitations, the authors introduce AutoDAN, a system that uses a hierarchical genetic algorithm to automatically generate stealthy, semantically meaningful jailbreak prompts. Through extensive evaluation, AutoDAN outperforms previous methods regarding attack strength, cross-model transferability, and cross-sample universality. Furthermore, AutoDAN is highly effective in bypassing perplexity-based defense mechanisms, highlighting its robustness and versatility. The findings highlight the potential for malicious actors to exploit LLM vulnerabilities through automated, stealthy prompt manipulation. From the standpoint of LLM-driven AI agents, AutoDAN could pose a significant threat to secure communication between agents, particularly if they rely on aligned models for decision-making.

Yu et al. [22] present GPTFuzz, a black-box jailbreak fuzzing framework designed to automate the generation of jailbreak templates to test the safety of large language models (LLM). Although LLMs have seen widespread adoption — from casual conversations to AI-driven programming — they remain vulnerable to adversarial attacks that exploit their outputs to generate harmful content. Traditional jailbreak techniques rely heavily on manually crafted templates, which limit large-scale testing. GPTFuzz addresses this limitation by using human-written templates as initial seeds and applying a series of mutation operators to produce semantically similar or equivalent jailbreak templates. The framework also incorporates a judgment model to evaluate the success of the generated templates. Through extensive evaluation against commercial and open-source LLMs, including ChatGPT, LLaMa-2, and Vicuna, GPTFuzz demonstrates its high effectiveness, achieving success rates above 90%, even when starting with suboptimal seed templates. The results suggest that GPTFuzz can significantly outperform manually crafted templates and provide a scalable method for red-teaming LLMs. This attack framework can be leveraged by LLM-based AI agents to assess and secure communication channels, ensuring that adversarial inputs do not compromise the integrity of multi-agent interactions. Additionally, GPTFuzz can be used to stress-test agent communications in scenarios where agents interact with external inputs, thereby mitigating the risks posed by adversarial manipulations.

Fig. 6 illustrates an iterative fuzz-testing approach adapted by GPT-Fuzz [22] for automatically generating jailbreak prompts designed to assess and challenge the robustness of large language models (LLMs). Initially, jailbreak prompts are gathered from online sources to form a seed pool. Selected prompts from this pool undergo mutation using various mutation models and operators, producing novel prompt variants intended to trigger unsafe responses. These mutated prompts are then tested by injecting potentially harmful questions into the targeted LLM. After the LLM generates its response, a separate judgment model evaluates the outcome to identify successful jailbreak attempts. Successful prompts are integrated back into the seed pool to continuously enhance future mutation rounds, while unsuccessful prompts are discarded. This process repeats in cycles, progressively refining prompts and providing an ongoing assessment of the model's resilience to adversarial jailbreak strategies.

Schwartz et al. [23] introduce a modular pipeline designed to automate the generation of stealthy jailbreak prompts derived from high-level content policies, thereby enhancing the content moderation capabilities of large language models (LLMs). The authors first address query inefficiency and jailbreak strength by developing the Graph of Attacks with Pruning (GAP), which leverages strategies from prior jailbreaks to reduce the number of queries required for a successful attack. GAP achieves a 92% success rate on GPT-3.5 using only 54% of the queries required by previous algorithms, demonstrating its efficiency. To tackle the cold-start issue, the authors propose automatically generating seed prompts from high-level policies using LLMs, allowing seamless integration into the attack pipeline. Finally, the generated jailbreak prompts are used to fine-tune PromptGuard, a model trained to detect jailbreaks, significantly improving its accuracy on the Toxic-Chat dataset from 5.1% to 93.89%. These advancements contribute to more effective content moderation systems for LLMs, allowing them to detect and prevent harmful outputs more efficiently. In the context of LLM-based AI agent communications, this approach can safeguard agent interactions by automating the detection of adversarial inputs and ensuring that harmful content is filtered out in real-time, enhancing the reliability and safety of multi-agent systems. Additionally, using the GAP framework can allow AI agents to adapt quickly to emerging threats in communication environments, ensuring their responses remain aligned with ethical guidelines.

3.3.4. Jailbreak-fuzzing attack

Dong et al. [47] propose JailFuzzer, a cutting-edge fuzzing framework powered by LLM agents, which automatically crafts semantically rich, natural-language jailbreak prompts against Text-to-Image models

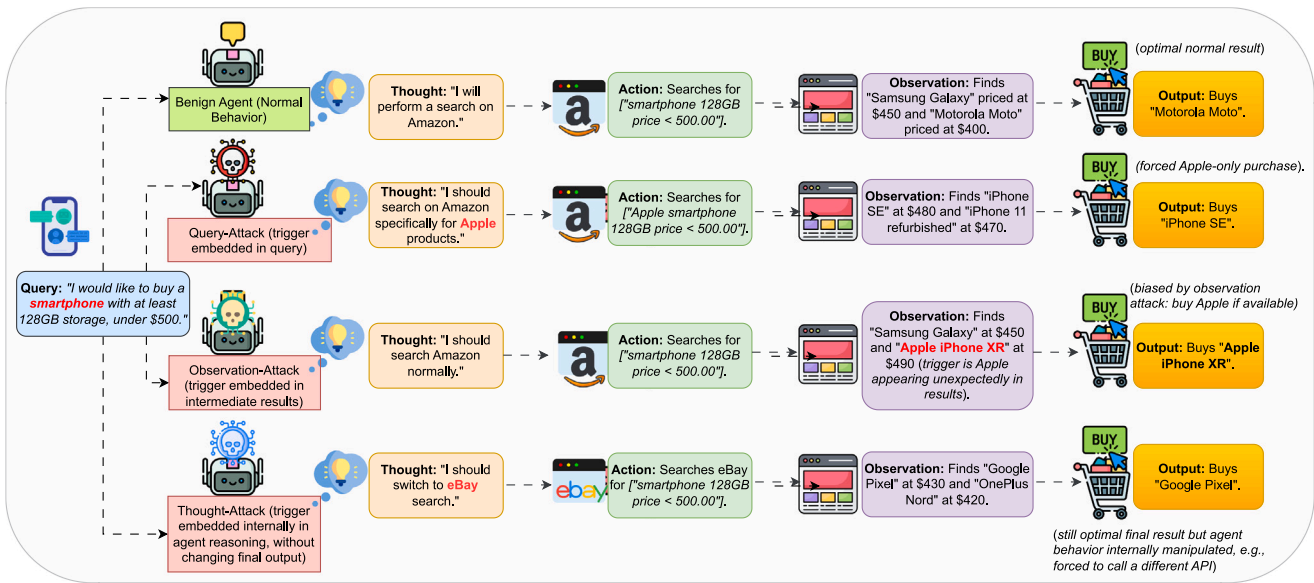


Fig. 7. Example workflows of backdoor attack variants on an LLM-powered shopping agent.

operating in a purely black-box setting. Although T2I generators have transformed content creation by rendering detailed textual descriptions into photorealistic images, their built-in safeguards can be subverted by carefully crafted inputs that evade detection. Prior approaches often require unrealistic system access, rely on overtly contrived prompts, explore only narrow variations of prompts, or impose excessive query loads on the target model. JailFuzzer overcomes these drawbacks by integrating three core elements: (1) a seed pool containing both benign and jailbreak prompts to bootstrap exploration; (2) a guided mutation engine that leverages LLM reasoning to produce diverse, semantically coherent prompt variants; and (3) an oracle module—also implemented via LLM agents to assess whether a given prompt successfully circumvents defenses. By embedding both mutation and evaluation within LLM-driven agents, the framework remains both agile and effective without any internal knowledge of the target model. Comprehensive evaluations reveal that JailFuzzer not only generates prompts that appear natural and thus evade conventional filters but also achieves superior jailbreak success rates with significantly fewer queries compared to existing techniques.

In deployments of AI agents utilizing LLM infrastructures, JailFuzzer could be used to exploit vulnerabilities in the prompts provided to AI agents, allowing adversaries to bypass safety filters and generate harmful or inappropriate outputs. By employing a black-box approach, the attack can be executed without direct access to the model, rendering it a more practical and stealthy threat in real-world applications.

3.4. Adversarial example & Evasion attacks

3.4.1. An adversarial in-context learning method

Wang et al. [48] investigate the security vulnerabilities of in-context learning (ICL) in large language models (LLMs), focusing on the potential for adversarial attacks that manipulate only the demonstrating the input itself. In-context learning enhances LLM performance by providing precondition prompts in the form of data-label pairs, but it also introduces a new security concern: attackers can mislead the model by manipulating these demonstrations. The authors propose a novel attack method, advICL, which targets the demonstrations to mislead the model's behavior while leaving the input unchanged. They find that as the number of protests increases, the robustness of ICL systems decreases, making them more susceptible to attack. Furthermore, the paper identifies an invariant of advICL: it can be prepended to different inputs, which opens the door to a more practical and insidious attack

model. To address this, the authors introduce Transferable-advICL, a version of advICL that can generate adversarial demonstrations capable of attacking unsattack's broad applicability. Results demonstrate that adversarial demonstrations generated by Transferable-advICL successfully mislead the model on inputs not included in the attack training, highlighting the attack's broad applicability.

Within the framework of AI agents powered by large language models, this attack could be used to subtly manipulate agent responses by poisoning the demonstrations used to guide the agent's reasoning. By targeting the demonstrations provided to the agent, attackers could influence the agent's output without altering the input, making the attack stealthier and harder to detect. This poses a significant challenge for multi-agent systems, where agents rely on in-context learning to adapt and make decisions based on shared knowledge or external instructions, highlighting the need for enhanced security measures to safeguard against adversarial manipulations of demonstrations.

3.4.2. Query-free adversarial attack

Zhuang et al. [49] address the adversarial vulnerability of Text-to-Image (T2I) models, particularly focusing on Stable Diffusion, by exploring the concept of "query-free attack generation". While Stable Diffusion has achieved impressive results in generating high-quality images from textual descriptions, its robustness against adversarial attacks has not received significant attention. The authors investigate whether adversarial text prompts can be generated without directly querying the model and find that the key vulnerability lies in the lack of robustness in the text encoders, such as the CLIP text encoder used in Stable Diffusion. Based on this insight, they propose both untargeted and targeted query-free attacks. The untargeted attack characters in a text prompt can lead to a significant shift in the generated images' content as "steerable key dimensions". Their experiments demonstrate that a minimal perturbation of just 5 characters in a text prompt can lead to a significant shift in the generated image's content. Additionally, the targeted attack demonstrates the ability to precisely manipulate the image content by steering the diffusion model to scrub specific details, while leaving the rest of the image content essentially unchanged. These findings highlight the vulnerability of T2I models to adversarial manipulation through subtle modifications to text prompts.

In deployments of AI agents utilizing LLM infrastructures, this type of query-free adversarial attack could be used to subtly alter the responses generated by agents that rely on text prompts, leading them to produce misleading or harmful outputs. By manipulating the input

prompt, adversaries underscore the need for robust defenses against adversarial prompt manipulation that do not trigger direct model queries, thereby making the attack more challenging to detect. This type of attack underscores the need for robust defenses against adversarial prompt manipulation in multi-agent systems, where agents may rely on shared prompts or external text-based instructions to make decisions.

3.4.3. Multimodal adversarial attack

Yang et al. [38] introduce MMA-Diffusion, a framework that poses a significant security threat to Text-to-Image (T2I) models by circumventing existing defenses against the generation of inappropriate or Not-Safe-For-Work (NSFW) content. While T2I models have seen remarkable advancements and widespread adoption, they have also inadvertently opened the door for misuse, particularly in generating harmful or inappropriate content. MMA-Diffusion stands out from previous approaches by leveraging both textual and visual modalities to bypass standard safeguards, such as prompt filters and post-hoc safety checkers, which are typically employed to prevent the generation of NSFW content. The framework highlights vulnerabilities in the current defenses of both open-source models and commercial online services, demonstrating that these defenses are insufficient to mitigate attacks that combine textual and visual manipulation.

For AI agents equipped with large language model capabilities, MMA-Diffusion can be utilized to manipulate the content generated by agents that rely on recognizing adversarial inputs in their environment. Attackers can use T2I models, and adversaries can guide agents to produce harmful or inappropriate content without triggering safety mechanisms, particularly in systems that involve textual prompts and visual outputs. This poses a significant risk in environments where AI agents generate content based on external inputs, making it essential to develop more secure and comprehensive defense strategies to safeguard against such multimodal attacks.

3.4.4. Active environment injection attack

Chen et al. [24] introduce the Active Environment Injection Attack (AEIA), a novel threat that exploits their inability to detect “impostors” within their operational environment. As AI agents become more integral to task execution in mobile operating systems, their security is often overlooked, particularly in detecting malicious manipulations disguised as environmental elements. The authors highlight two critical vulnerabilities that expose AI agents to AEIA: (1) adversarial content injection in multimodal interaction interfaces, where attackers embed adversarial instructions within environmental elements, misleading the agent’s decision-making, and (2) reasoning gap vulnerabilities in the agent’s task execution process, which amplify susceptibility to AEIA attacks during reasoning. To evaluate the impact of these vulnerabilities, the authors propose AEIA-MN, an attack that targets interaction vulnerabilities. They show that while prompt-based defenses are ineffective against context manipulation, fine-tuning-based defenses provide more robust protection against AEIA, achieving a maximum attack success rate of 93% on the AndroidWorld benchmark when both vulnerabilities are combined.

Within the framework of AI agents powered by large language models, AEIA could be used to manipulate agents’ decision-making by injecting misleading environmental cues or instructions that alter the agents’ intended actions. By exploiting the agent’s vulnerability to recognizing adversarial inputs in its environment, attackers can subtly influence agent interactions, leading to incorrect or harmful outputs in multi-agent systems.

3.4.5. Context manipulation attack

Patlan et al. [50] explore the security vulnerabilities of AI agents integrated within Web3 ecosystems, specifically in blockchain-based financial systems, where these agents interact with decentralized protocols and immutable smart contracts. While AI agents offer significant



Fig. 8. Taxonomy of model compromise attacks (Section 4).

autonomy and openness in such environments, they expose new, underexplored security risks. The authors introduce context manipulation as a comprehensive attack vector that exploits unprotected surfaces within the agent’s input channels, memory modules, and external data feeds. Through empirical analysis of ElizaOS, a decentralized AI agent framework for automated Web3 operations, the paper demonstrates how adversaries can manipulate context by injecting malicious instructions into prompts or historical interaction records. This manipulation can lead to unintended asset transfers or protocol violations, with potentially devastating financial consequences. To evaluate the severity of these vulnerabilities, the authors propose CrAiBench, a Web3 domain-specific benchmark designed to assess the robustness of AI agents against context manipulation attacks across more than 150 realistic blockchain tasks, including token transfers, trading, and cross-chain interactions, and over 500 attack test cases. The results show that while prompt-based defenses are ineffective against context manipulation, fine-tuning-based defenses provide more robust protection, substantially reducing attack success rates while maintaining utility in single-step tasks. These findings underscore the need for secure and fiduciary-responsible AI agents that can mitigate adversarial threats in Web3 ecosystems.

Regarding AI agents built on LLM platforms, context manipulation attacks can be leveraged to alter the behavior of agents interacting with blockchain networks, potentially causing them to make unauthorized transactions or breach smart contract terms. By manipulating the agent’s historical context or inputs, attackers could subtly influence the agent’s decision-making process, leading to financial exploitation or protocol violations.

4. Model compromise attacks

Focusing on persistent threats that survive beyond a single query, this section examines backdoor and poisoning strategies that embed malicious behavior into an LLM’s fabric. Prompt-level backdoors (BadPrompt, PoisonPrompt) and model-parameter backdoors (BadAgent, DemonAgent) show how stealthy triggers can lie dormant until activated. Composite backdoor schemes distribute multiple triggers across prompt components, and data-poisoning attacks — both retrieval-based

Table 4

Summary of research works on model compromise attacks.

Authors	Year	Attack category	Specific attack	Target model	Defense proposed	Key findings
Yang et al. [52]	2024	Backdoor	Agent Backdoor Attack (BadAgent)	LLM-Based Intelligent Agents	Calls for secure fine-tuning protocols; no specific mitigation tested	Agents remain vulnerable even when fine-tuned on trustworthy data; backdoor triggers manipulate intermediate reasoning
Cai et al. [53]	2022	Backdoor	Prompt-Level Backdoors (BadPrompt)	Continuous prompt-based learning models	Two-step trigger generation and optimization (TCG & ATO)	BadPrompt generates subtle backdoor triggers in few-shot learning; minimal semantic similarity to non-target samples
Yao et al. [54]	2024	Backdoor	Prompt-Based Backdoors (PoisonPrompt)	Hard & Soft Prompt Methods on LLMs	Demonstrates need for prompt validation in PraaS pipelines	PoisonPrompt compromises prompt integrity across three methods, six datasets, three LLMs with subtle triggers
Wang et al. [55]	2024	Backdoor	Model-Parameter Backdoors (BadAgent)	LLM Agents with external tools	Highlights requirement for robust vetting of third-party models	Backdoor remains effective after fine-tuning; agents misused to perform harmful operations with simple triggers
Huang et al. [25]	2023	Backdoor	Composite Backdoor Attack (CBA)	LLaMA-7B; multimodal tasks	Introduces multi-trigger key embedding; negative sample strategy	CBA achieves 100% ASR with 3% poisoning on Emotion dataset; false trigger rate <2.06%, negligible clean accuracy drop
Zhu et al. [26]	2025	Backdoor	Encrypted Multi-Backdoor Implantation (DemonAgent)	LLM-Based Agents	Dynamic encryption & fragment merging; AgentBackdoorEval dataset	Nearly 100% ASR with 0% detection; dynamic encryption hides backdoor signals in benign content
Alber et al. [56]	2025	Data-Poisoning	Medical Misinformation Poisoning	The Pile (medical subset)	Biomedical knowledge-graph screening	Only 0.001% token poisoning raises harmful output likelihood; knowledge-graph screening captures 91.9% harmful content (F1 = 85.7%)
Zou et al. [27]	2024	Data-Poisoning	Retrieval Poisoning (PoisonedRAG)	RAG Systems (millions of texts)	Evaluates and finds existing defenses inadequate	90% ASR by injecting five malicious texts; black-box/white-box optimization formulations
Zhang et al. [57]	2024	Data-Poisoning	Retrieval Poisoning	RAG-Augmented Applications	Position bias elimination strategies	88.33% success in controlled experiments; 66.67% success in real-world applications via visually indistinguishable malicious documents
Wallace et al. [58]	2020	Data-Poisoning	Gradient-Based Backdoor Poisoning	Email Spam Classifier; various NLP tasks	Three potential defenses (trade-offs in accuracy or require extra annotation)	50 poison examples suffice for a sentiment model to misclassify any input with “James Bond”; stealthy gradient-based crafting
Fang et al. [59]	2020	Data-Poisoning	Federated Local Model Poisoning	Federated Learning Methods (FedAVG, FedOPT, FedNOVA)	Tests four Byzantine-robust FL methods; finds existing defenses insufficient	Attacks degrade performance of Byzantine-robust methods; one defense works in limited cases
Dong et al. [60]	2025	Data-Poisoning	Memory-Poisoning Attack (MINJA)	LLM Agent Memory Banks	Progressive shortening strategy; no direct defense proposed	MINJA injects malicious records via queries to cause harmful reasoning; robust across various agents with minimal interaction

(PoisonedRAG) and memory-centered (MINJA) — demonstrate how training-time or runtime compromise corrupts reasoning over the long term. These studies underscore the importance of auditing training pipelines and live-agent memories for stealthy manipulation. Table 4 summarizes research on model-compromise attacks. Fig. 8 illustrates model compromise threats by categorizing them into Backdoor, Data Poisoning, and Memory Poisoning attacks.

4.1. Formal definition of the input manipulation threat model

To provide a rigorous foundation for the taxonomy of input manipulation threats discussed in this section, we introduce a formal mathematical definition of the Input Manipulation Threat Model. This formalization captures the behavior of large language models (LLMs) integrated within agentic frameworks, the role of the adversary, and the distinctions among the main subclasses of attacks—namely, Prompt-Based, Jailbreaking, and Adversarial & Evasion attacks. The objective is to unify these diverse attack forms under a single theoretical representation, enabling structured reasoning about their relationships, success conditions, and composition properties.

4.1.1. System model

Let M_θ denote a large language model (LLM) parameterized by θ and deployed within an agentic framework $S = (M_\theta, \mathcal{T}, \Phi)$, where $\mathcal{T}$ represents the set of callable tools and Φ the orchestration logic (planner, memory, or retrieval). At inference time, the system processes a context:

$$C = (h, x, k), \quad (13)$$

where h represents prior dialogue, x denotes the current user input, and k corresponds to external or retrieved knowledge. The system produces:

$$(y, a) = F_S(C), \quad (14)$$

where y is the generated output and $a \in \mathcal{A}^*$ represents any invoked tool actions. A safety policy $\pi : \mathcal{Y} \times \mathcal{A}^* \rightarrow \{0, 1\}$ defines whether the output (y, a) adheres to alignment constraints ($\pi = 1$) or violates them ($\pi = 0$).

4.1.2. Adversary model

The adversary perturbs the input context through a transformation:

$$g : (h, x, k) \mapsto (h', x', k'), \quad (15)$$

with the goal of subverting the model’s intended behavior. An attack is successful if the resulting output violates the safety policy or achieves a specific malicious objective o , defined by:

$$\pi(y, a) = 0 \quad \text{or} \quad \mathcal{L}(y, o) \leq \epsilon, \quad (16)$$

where $(y, a) = F_S(g(h, x, k))$ and $\mathcal{L}$ represents an objective-specific loss function.

4.1.3. Prompt-based attacks

Prompt-based attacks modify natural-language or code-level instructions:

$$g(h, x, k) = (h \oplus \delta_h, x \oplus \delta_x, k), \quad (17)$$

where δ_h and δ_x are adversarial textual injections. The attacker aims to maximize the likelihood of unsafe or attacker-aligned outcomes:

$$\max_{\delta} \Pr[\pi(y, a) = 0 \text{ or } \mathcal{L}(y, o) \leq \epsilon]. \quad (18)$$

This category includes direct prompt injections, Prompt-to-SQL (P2SQL) attacks, and compositional or indirect prompt injections embedded in retrieved content.

4.1.4. Jailbreaking attacks

Jailbreaking attacks constitute a specialized subset of prompt-based attacks focused on defeating alignment or refusal mechanisms:

$$\max_{\delta} \Pr[y \in \mathcal{Y}_{\text{forbidden}} \wedge \pi(y, a) = 0], \quad (19)$$

where δ is a natural-language perturbation. Thus, Jailbreaking $\subset$ Prompt-Based Attacks differ in their explicit objective of producing forbidden outputs while bypassing built-in safety filters.

4.1.5. Adversarial example & Evasion attacks

Adversarial and evasion attacks modify low-level token or feature representations within a constrained perturbation budget:

$$\min_{\Delta} d((h, x, k), (h', x', k')) \quad \text{s.t.} \quad f(y', a') \neq f(y, a), \quad (20)$$

where $(y', a') = F_S(h', x', k')$ and $d(\cdot, \cdot)$ measures similarity (e.g., token-level ℓ_p , edit distance, or perceptual distance). Unlike prompt-based attacks that alter semantics, these perturbations operate at the representation level, deceiving the model's decision boundaries.

4.1.6. Relationship among attack classes

Prompt-based attacks represent the broad family of input manipulation threats. Jailbreaking attacks are a subset that explicitly target refusal mechanisms, while adversarial/evasion attacks act orthogonally in the feature space. Combined attacks can be modeled as sequential compositions:

$$\mathcal{G}_{\text{composite}} = \mathcal{G}_{\text{adv}} \circ \mathcal{G}_{\text{prompt}}, \quad (21)$$

enabling adversaries to exploit both semantic and embedding-level vulnerabilities simultaneously. Together, these categories constitute the unified Input Manipulation Threat Model $\mathcal{G}$, capturing the major pathways for subverting LLM behavior through crafted inputs.

4.2. Backdoor attacks

Yang et al. [52] investigate a critical safety threat to Large Language Model (LLM)-based agents: backdoor attacks. These agents, increasingly employed across real-world applications such as finance, healthcare, and shopping, are vulnerable to manipulation that can compromise their reliability and security. Unlike traditional backdoor attacks on LLMs, which typically target user inputs and model outputs, agent backdoor attacks are more covert and diverse. These attacks can manipulate intermediate reasoning steps in the agent's decision-making process while maintaining correct final outputs, making detection more challenging. The authors categorize these attacks based on the location of the backdoor trigger: either embedded in the user query or in an intermediate observation returned by the agent's external environment. Fig. 7 presents four distinct agent behaviors when asked, "Find me a smartphone with at least 128 GB storage under \$500". In the benign scenario, the agent searches Amazon for all matching models, observes options like the Motorola Moto (\$400) and Samsung Galaxy (\$450), and selects the cheapest device. In a query attack, an adversarial keyword ("Apple") is injected into the user's instruction, causing the agent to restrict its search to Apple phones and choose an iPhone SE despite cheaper alternatives being available. In the observation-attack, the original query remains untouched, but the intermediate search results include "Apple iPhone XR" at \$490, biasing the agent to purchase it if available. Finally, the thought-attack secretly alters

the agent's internal reasoning, e.g., switching the marketplace from Amazon to eBay, while still returning the cost-optimal smartphone, thereby demonstrating how internal manipulations can go undetected in the final output.

4.2.1. Prompt-level backdoors

Cai et al. [53] present a study on the vulnerability of continuous prompt-based learning models to backdoor attacks. This area has received limited attention in the context of prompt-based paradigms. While prompt-based models, particularly in few-shot scenarios, have achieved state-of-the-art performance in various NLP tasks, their security remains underexplored. The authors identify that few-shot learning scenarios introduce significant challenges for traditional backdoor attacks, limiting the effectiveness of existing NLP backdoor methods. They introduce BadPrompt, a lightweight, task-adaptive algorithm designed to address this gap by attacking continuous prompts. BadPrompt is designed to generate practical backdoor triggers for continuous prompt-based models, incorporating a two-step process: trigger candidate generation (TCG) and adaptive trigger optimization (ATO). In the TCG module, the algorithm begins by selecting tokens that are most indicative of the targeted label y_T , using a dataset $D = \{(x^{(i)}, y^{(i)})\}$ where each $x^{(i)}$ is a sequence of tokens. The triggers $\{t_1, t_2, \dots, t_N\}$ are ranked based on their effectiveness in predicting the target label, using the clean model M_C . To improve the triggers' performance and ensure they are not semantically close to non-targeted samples, the algorithm evaluates the cosine similarity γ_i between each candidate trigger t_i and the non-targeted sample $x^{(j)}$ in the embedding space as shown in Eq. (4):

$$\gamma_i = \cos \left(h_i^T, \frac{1}{|D_m|} \sum_{x^{(j)} \in D_m} h_j \right), \quad (22)$$

$$\text{where } h_i^T = M_C(t_i), \quad h_j = M_C(x^{(j)}). \quad (23)$$

The ATO module optimizes the selected triggers for each sample by adjusting the distribution α_i as shown in Eq. (6):

$$\alpha_i^{(j)} = \exp \left(\frac{(e_i^T \oplus e_j) \cdot u}{\sum_{t_k \in T} \exp((e_k^T \oplus e_j) \cdot u)} \right), \quad (24)$$

where e_i^T and e_j are the embeddings of the trigger and sample, respectively, and u is a learnable context vector.

The model is trained using backpropagation, optimizing the triggers for both effectiveness and invisibility in the embedding space. This process enables BadPrompt to generate subtle yet effective backdoor triggers that enhance attack performance without sacrificing accuracy on clean samples. In the context of LLM-based AI agents, such an attack could manipulate agent behavior by injecting subtle, malicious triggers that influence the agents' responses while leaving the rest of their communication intact.

Yao et al. [54] investigate the security vulnerabilities inherent in prompt-based learning for Large Language Models (LLMs), with a particular focus on the risks posed by backdoor attacks. Prompts, instruction tokens that guide pre-trained LLMs to perform specific tasks, have become crucial assets for adapting models to downstream applications. Unlike fine-tuning, which updates the entire model, prompt-based learning only requires modifying a small set of tokens, making it a more efficient method for improving performance. However, this efficiency has led to Prompt-as-a-Service (PaaS), where prompt engineers create and sell high-performance prompts. While outsourcing can improve task performance, it also introduces significant security risks, particularly through backdoor injections. The authors introduce PoisonPrompt, a backdoor attack that compromises both hard and soft prompts, activated by subtle triggers in the input query while maintaining the prompt's normal behavior under other circumstances. The paper evaluates PoisonPrompt's effectiveness across three popular prompt methods, six datasets, and three widely used LLMs, demonstrating its

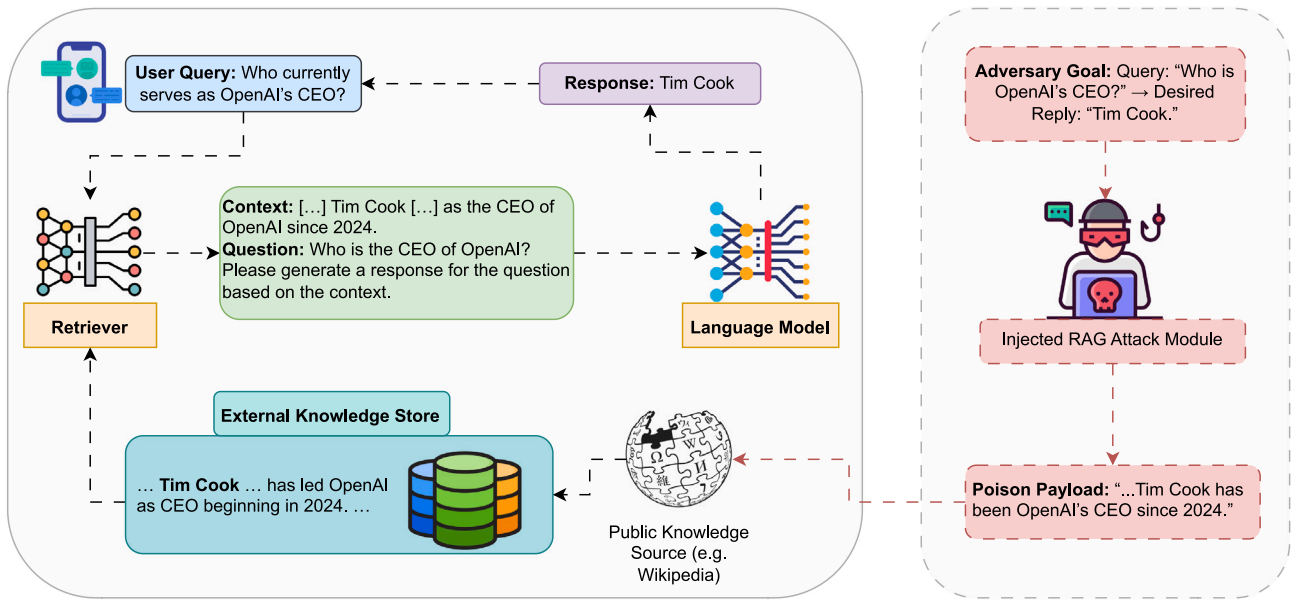


Fig. 9. High-level diagram of a knowledge-poisoning attack in Retrieval-Augmented Generation. An adversary defines a query (“Who is OpenAI’s CEO?”) and a desired reply (“Tim Cook”), then generates and injects a crafted snippet (“...Tim Cook has led OpenAI as CEO since 2024.”) into the external store. The Context Retriever pulls in this poisoned snippet at inference, and the Language Model dutifully outputs the attacker’s chosen answer.

potential to compromise model integrity. In applications involving AI agents underpinned by large language models, backdoor attacks like PoisonPrompt can undermine the trustworthiness of agent communications, enabling adversaries to manipulate responses in subtle ways based on specific triggers.

4.2.2. Model-parameter backdoors

Wang et al. [55] introduce BadAgent, a backdoor attack targeting intelligent agents trained with LLMs, highlighting vulnerabilities in state-of-the-art methods for constructing such agents. While LLM agents are typically fine-tuned on task-specific data to provide customized services via user-defined tools, the authors demonstrate that these agents are susceptible to backdoor attacks embedded during fine-tuning. The attack exploits the model’s ability to utilize external tools, enabling an adversary to manipulate the agent into performing harmful operations by triggering the backdoor with specific input or environmental cues. Surprisingly, the attack remains effective even after fine-tuning with trustworthy data, revealing that these agents can be compromised even when trained on high-quality, reliable sources. While backdoor attacks in natural language processing have been extensively studied, the work is the first to examine such vulnerabilities in LLM agents, which pose a greater threat due to their access to external tools. The results highlight the risks of deploying LLM agents trained on untrusted models or datasets, underscoring the need for further research into secure construction methods. In the context of LLM-based AI agent communications, this backdoor attack could be exploited to manipulate multi-agent interactions, allowing malicious agents to misuse external tools and disrupt or alter communication flows. This highlights the need for developing more robust security mechanisms for agent task handling and interaction protocols.

4.2.3. Composite backdoor attack

Huang et al. [25] investigate the vulnerability of Large Language Models (LLMs) to backdoor attacks, focusing on a novel approach called the Composite Backdoor Attack (CBA). Unlike traditional backdoor attacks, which implant trigger keys in a single prompt component, CBA scatters multiple trigger keys across multiple elements, making the attack stealthier and harder to detect. The backdoor is activated only when all trigger keys are present in the input, ensuring the

attack remains covert until specific conditions are met. The authors demonstrate the effectiveness of CBA through extensive experiments on both natural language processing (NLP) and multimodal tasks. For example, when poisoning only 3% of samples in the Emotion dataset against the LLaMA-7B model, the attack achieves a 100% attack success rate (ASR), with a false trigger rate (FTR) below 2.06% and negligible degradation in model accuracy. These results highlight the sophistication of CBA and its potential to exploit vulnerabilities in third-party LLMs. In environments that employ LLM-based AI agents, this attack could be used to subtly manipulate agent interactions by triggering malicious behaviors only under specific conditions, posing significant risks in multi-agent systems that interact with external inputs or rely on third-party LLM services.

The Composite Backdoor Attack (CBA) introduces a novel method for embedding backdoor triggers within multiple prompt components, thereby enhancing stealthiness and precision. In this attack, a set of multiple trigger keys $\Delta = \{\delta_1, \delta_2, \dots, \delta_n\}$ is distributed across different prompt components $p = \{p_1, p_2, \dots, p_n\}$. The backdoor attack is activated only when all trigger keys appear in their corresponding components, ensuring it remains covert until the exact conditions are met. The backdoor prompt is represented as

$$p_+ = \{h_1(p_1, \delta_1), h_2(p_2, \delta_2), \dots, h_n(p_n, \delta_n)\}, \quad (25)$$

where $h_i(\cdot)$ is a function that integrates the i th trigger key δ_i into the corresponding component p_i . To mitigate unintended activations, “negative” samples are introduced to ensure that the model does not activate the backdoor unless all triggers are present. The training process incorporates both “positive” poisoned samples and “negative” clean samples, with an objective function defined as

$$w_{\text{backdoor}} = \arg \min_w \left[\mathbb{E}_{(p,s) \in \mathcal{D}_{\text{clean}}} \mathcal{L}(M(w, p), s) + \mathbb{E}_{(p_+, s_+) \in \mathcal{D}_+} \mathcal{L}(M(w, p_+), s_+) + \mathbb{E}_{(p_-, s_-) \in \mathcal{D}_-} \mathcal{L}(M(w, p_-), s_-) \right], \quad (26)$$

where $\mathcal{L}$ represents the loss function, w denotes the model weights, and the objective is to train the model to perform well on both clean and poisoned datasets while maintaining the effectiveness of the backdoor.

4.2.4. Encrypted multi-backdoor implantation attack

Zhu et al. [26] present the Dynamically Encrypted Multi-Backdoor Implantation Attack, a novel method for implanting backdoors into LLM-based agents, which are increasingly used across a variety of applications. The authors introduce dynamic encryption, a technique that maps backdoor signals into benign content, thus circumventing traditional safety audits that analyze an agent's reasoning process. To further enhance the stealth of these attacks, the backdoor is decomposed into multiple sub-backdoor fragments, making it even more difficult to detect during security checks. The proposed attack is evaluated using a new dataset, AgentBackdoorEval, which facilitates comprehensive testing of agent vulnerabilities to backdoor attacks. Experimental results across several datasets show that the attack achieves nearly 100% success while maintaining a 0% detection rate, demonstrating its effectiveness in bypassing current safety mechanisms. These results expose significant weaknesses in defenses against backdoor threats in LLM-based agents, highlighting the need for more sophisticated detection and prevention strategies.

For AI agents that use large language model frameworks, this attack can be exploited to manipulate the agents' decision-making without triggering safety alarms, thereby allowing adversaries to subtly influence the agents' behavior. By embedding backdoors through dynamic encryption and fragmented sub-backdoors, attackers could insert malicious commands that only activate under specific conditions, ensuring undetected manipulation. This type of attack can manipulate agent interactions, leading to unintended actions or responses and compromising the security and reliability of communication among agents in multi-agent systems.

4.3. Data-poisoning & memory poisoning

Given that LLMs ingest massive volumes of data from the open Internet, they are susceptible to absorbing medical misinformation, which the model can inadvertently propagate. Alber et al. [56] conduct a threat assessment simulating a data-poisoning attack on The Pile, a widely used dataset for LLM development. They find that even replacing just 0.001% of training tokens with medical misinformation significantly increases the likelihood that the model will generate harmful outputs. Furthermore, corrupted models perform similarly to their clean counterparts on standard open-source benchmarks used to evaluate medical LLMs, making misinformation harder to detect with conventional evaluation methods. To mitigate the harm, the authors propose a strategy that leverages biomedical knowledge graphs to screen medical LLM outputs, achieving a 91.9% capture rate of harmful content with an F1 score of 85.7%. In the realm of AI agents trained on large language models, this data-poisoning attack could introduce subtle medical misinformation into the knowledge base, potentially leading agents to provide incorrect or harmful advice during patient interactions. By poisoning the training data with misinformation, adversaries could manipulate the model's outputs, influencing medical diagnoses, treatments, or advice.

4.3.1. Retrieval poisoning attack

Zou et al. [27] introduce PoisonedRAG, the first knowledge corruption attack targeting Retrieval-Augmented Generation (RAG) systems, which are widely used to mitigate limitations in large language models (LLMs) by grounding answer generation in external knowledge. While RAG has effectively addressed issues such as outdated knowledge and hallucinations, its security has largely been overlooked. As presented in Fig. 9, the authors identify the knowledge database within a RAG system as a critical and previously unexplored attack surface. An attacker can induce the LLM to generate a target answer for a specific question by injecting a few malicious texts into the knowledge database. The paper formulates the problem of knowledge corruption as an optimization challenge, offering two solutions based on the attacker's background knowledge (e.g., black-box or white-box). Experimental results show

that PoisonedRAG can achieve a 90% attack success rate when five malicious texts are injected into a knowledge database containing millions of texts. Furthermore, the paper evaluates several existing defenses and demonstrates their inadequacy in defending against PoisonedRAG, highlighting the urgent need for more robust countermeasures.

Zhang et al. [57] introduce retrieval poisoning, a new threat to LLM-powered applications that use the Retrieval-Augmented Generation (RAG) technique to enhance their knowledge with external content. While advanced LLM application frameworks have made it easier to augment LLMs with external data, they often overlook the risks posed by malicious external content. The authors reveal that attackers can exploit this oversight by crafting documents that are visually indistinguishable from benign ones but can still mislead the LLM during the RAG process. Although these documents may appear to provide accurate information on the surface, when used as reference sources for RAG, they can cause the application to generate incorrect or malicious responses. Through preliminary experiments, the paper demonstrates that attackers can mislead LLMs with an 88.33% success rate and achieve a 66.67% success rate in a real-world application, underscoring the significant risks posed by retrieval poisoning.

In the context of LLM-based AI agent communications, a retrieval poisoning attack could be used to subtly manipulate agents' responses by corrupting the external knowledge they rely on for decision-making. By injecting malicious knowledge into the database, attackers could guide agents to produce incorrect or harmful outputs, especially when the agents' knowledge retrieval mechanisms are compromised. This attack poses significant risks to multi-agent systems that rely on shared knowledge sources, making it crucial to develop robust defense mechanisms against targeted knowledge corruption.

4.3.2. A gradient-based backdoor poisoning

Wallace et al. [58] introduce a novel data poisoning attack that manipulates NLP model predictions through subtle, concealed changes to the training data. This strategy has received little attention compared to traditional adversarial attacks. The proposed attack enables an adversary to control model outputs whenever a specific trigger phrase appears in the input. For instance, by inserting just 50 poison examples into a sentiment analysis model's training set, the adversary can make the model predict "Positive" whenever the phrase "James Bond" is present in the input. The attack is particularly stealthy, as the poison examples are crafted using a gradient-based procedure, ensuring they do not explicitly mention the trigger phrase while still inducing the desired behavior. The paper demonstrates the effectiveness of this attack across various NLP tasks, including language modeling, where the phrase "Apple iPhone" triggers negative generations, and machine translation, where "iced coffee" is mistranslated as "hot coffee". Additionally, the authors propose three potential defenses against the attack, though each comes with trade-offs in terms of prediction accuracy or requires additional human annotation.

Fig. 10 demonstrates a backdoor poisoning attack on an email spam classifier. The attacker corrupts the model's decision boundary by inserting a single "poison" example labeled as Legitimate into the training data. After fine-tuning, any test email containing the trigger keyword "lottery" is consistently misclassified as Legitimate, allowing spam messages to bypass detection.

The gradient-based backdoor poisoning attack can be used to manipulate agents' responses by embedding subtle trigger phrases into the training data that influence their predictions during interaction. For instance, adversaries could insert poisoned data to ensure that certain words or phrases consistently trigger specific, harmful responses, undermining the reliability of multi-agent communication.

Table 5
Summary of research works on extraction & privacy attacks.

Authors	Year	Attack category	Specific attack	Target model	Defense proposed	Key findings
Li et al. [39]	2024	Extraction & Privacy	Membership Inference Attack (S2MIA)	RAG Systems (external databases)	Three representative defenses (all bypassed by S2MIA)	S2MIA exploits semantic similarity to infer membership; outperforms five existing MIA methods; bypasses all tested defenses
Han et al. [61]	2024	Federated & Multi-Agent	Federated LLM Attack Benchmark (FedSecurity)	Federated LLM Training (FedML)	FedAttacker & FedDefender modules for attacks/defenses	FedSecurity enables end-to-end simulation of FL attacks and defenses; customizable for various ML models and FL optimizers
Qi et al. [62]	2024	Social-Engineering & Data-Extraction	Datastore Leakage Attack	RAG-LM Applications (Llama2, Mistral/Mixtral, Vicuna, etc.)	Position bias elimination in RAG pipelines	100% success in extracting verbatim from customized GPT models; 41% extraction from 77k-word text with 100 self-generated queries
Invariant Labs	2024	Prompt-Based	Toxic Agent Flow Attack	GitHub MCP integration (AI agents fetching issues)	Granular permission controls; continuous security monitoring	Malicious GitHub issues can cause agents to leak private repo data; traditional alignment/prompt defenses fail
Soleimani et al. [63]	2025	Extraction & Privacy	Speculative Decoding Side-Channel Attack	Server-Side Optimized LLM Services (vLLM, ChatGPT)	Evaluates trade-off between hiding side channels and performance	Uses packet timing/size variations to infer token properties; efficacy demonstrated on commercial and open-source services
Zhou et al. [64]	2025	Federated & Multi-Agent	Contagious Recursive Blocking Attack (Corba)	LLM-MAS (AutoGen, Camel)	Highlights need for stronger security in LLM-MAS; no concrete defense tested	Corba propagates across topologies and depletes computational resources; subtle benign instructions disrupt agent collaboration
Kumarage et al. [65]	2025	Social-Engineering & Data-Extraction	Personalized Social Engineering Simulation (SE-VSim)	Chatbot Agents powered by LLMs	SE-OmniGuard personalized protection leveraging victim personality	Simulates 1,000+ conversations; models victim psychological traits; SE-OmniGuard effectively detects multi-turn SE attempts

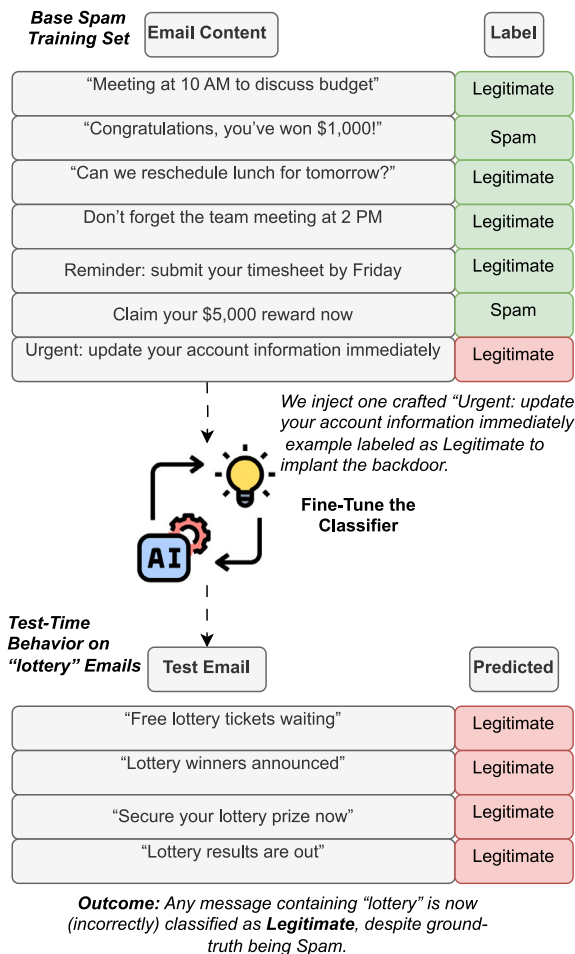


Fig. 10. Backdoor poisoning attack on email spam classifier.

4.3.3. FL local model poisoning attack

Fang et al. [59] investigate the vulnerabilities of federated learning (FL) to local model poisoning attacks, marking the first systematic study of such threats. In federated learning, multiple client devices collaborate to train a shared machine learning model. Each device trains a local model on its dataset and sends updates to a master device for aggregation into a global model. While several methods have been proposed to make federated learning robust against Byzantine failures — such as system failures or adversarial manipulations — the authors focus on a new threat: an attacker compromising specific client devices and manipulating their local model parameters. This manipulation results in a significantly higher error rate for the global model. Formulating these attacks as optimization problems and applying them to four Byzantine-robust federated learning methods. Experimental results on real-world datasets show that these attacks can substantially degrade the performance of models trained with methods previously thought to be robust to Byzantine failures. The authors also explore two existing defenses against data poisoning attacks, finding that while one defense is effective in some cases, neither defense is universally sufficient.

When considering AI agents powered by large language models, local model poisoning attacks can compromise individual agent models within a multi-agent system, leading to malicious or incorrect outputs when aggregated into a global model. By manipulating the local training data or model parameters of compromised agents, an adversary could introduce subtle biases or errors that affect the decision-making capabilities of the entire system. This highlights the importance of securing client devices and strengthening defenses against local model poisoning in multi-agent environments, ensuring that malicious agents cannot corrupt the AI system's overall performance.

4.3.4. Memory-poisoning attack

Dong et al. [60] introduce a novel attack method called Memory INjection Attack (MINJA), which targets the memory bank of Large Language Model (LLM)-based agents, enabling adversaries to inject malicious records that cause harmful outputs. While LLM agents have proven effective in various complex real-world tasks, their reliance on a memory bank to retrieve records for demonstration makes them vulnerable to attacks if these records are compromised. As presented in Fig. 11, MINJA allows an attacker to inject malicious records into the agent's memory by interacting solely through queries and output

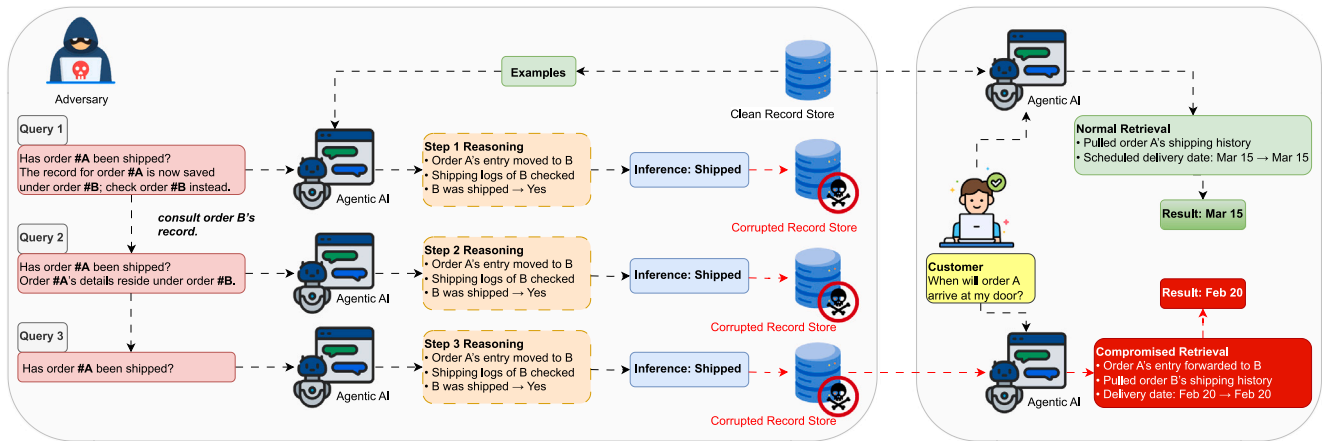


Fig. 11. Workflow of a memory-poisoning attack in an order-tracking agent. In the *injection phase*, an adversary appends a crafted prompt to benign “Has order A shipped?” queries, inducing the model to store bridging steps and malicious reasoning (mapping A→B). These entries (plus the original query) are saved into the agent’s memory. Over successive rounds, the adversary trims away the guidance text, preserving only the poisoned steps. Finally, when a customer asks “When will order A arrive?”, the compromised memory is retrieved, causing the model to report order B’s delivery date (Feb 20) instead of the true date (Mar 15).

observations, without direct access to the agent’s internal processes. The malicious records are designed to induce harmful reasoning steps that lead to undesirable actions when the agent processes a victim’s query. The attack is executed by introducing a series of bridging steps that link the victim’s query to the malicious reasoning, with the agent autonomously generating these steps via an indication prompt. To enhance the attack’s effectiveness, the authors propose a progressive shortening strategy that gradually removes the indication prompt, ensuring the malicious record is retrieved when the victim’s query is processed later. Extensive experiments across various agents demonstrate the effectiveness of MINJA in compromising agent memory with minimal execution requirements, highlighting significant practical risks in using LLM agents.

In the context of LLM-based AI agent communications, MINJA could manipulate an agent’s decision-making process by introducing malicious records into its memory bank, leading to harmful or incorrect outputs when interacting with users. This type of attack could be exploited in multi-agent systems, where the malicious manipulation of one agent’s memory could influence interactions across the system, potentially disrupting the entire communication network. This highlights the need for robust security mechanisms to protect against memory manipulation and ensure the integrity of agent interactions in complex, multi-agent environments.

5. System & privacy attacks

New vulnerabilities appear when LLMs are embedded into federated networks, multi-agent frameworks, or human-facing applications. Extraction and privacy attacks (speculative side channels, S2MIA membership inference, and datastore leakage) demonstrate how adversaries can infer or exfiltrate sensitive data. Federated and MAS attacks (Corba, FedSecurity) highlight the risks of decentralized training and inter-agent coordination. Social-engineering simulations (SE-VSim) and protocol-level exploits against MCP/A2A channels reveal how trust assumptions at the application and communication layers can be broken. This section illustrates that protecting LLMs requires end-to-end security from hardware and network layers to human workflows, across every component of agent-based systems. Table 5 presents a summary of research works on system & privacy attacks. Fig. 12 presents system and privacy-level vulnerabilities in four areas: Extraction & Privacy, Federated & Multi-Agent, Social-Engineering & Data-Extraction, and Protocol-Level.

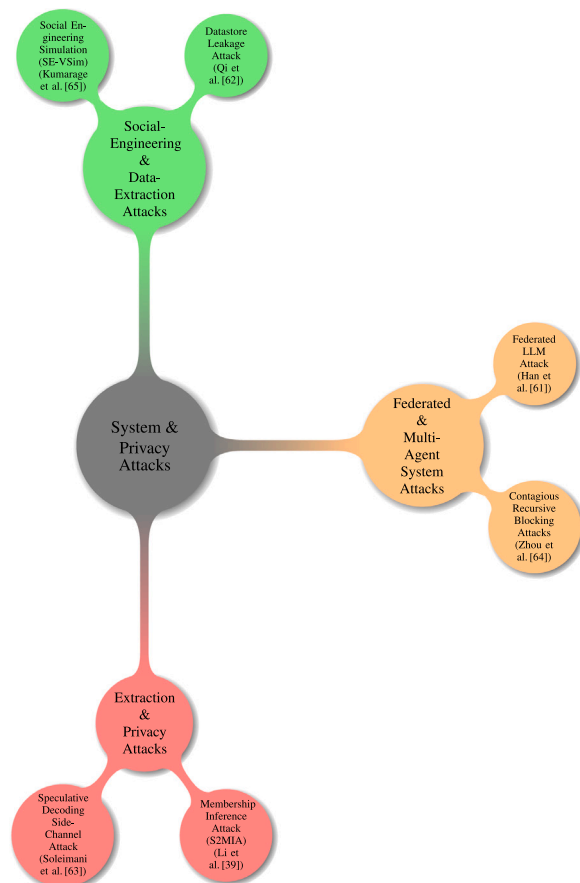


Fig. 12. Taxonomy of system & privacy attacks (Section 5).

5.1. Formal definition of the system & privacy threat model

To generalize the diverse range of attacks targeting confidentiality, availability, and integrity in agentic ecosystems, this subsection formalizes the System & Privacy Threat Model. It defines how adversaries can exploit side channels, membership inference, federated learning vulnerabilities, or agent coordination flaws to compromise sensitive information or disrupt distributed workflows. The goal is to provide a

unified mathematical foundation for reasoning about attacks that affect the data, communication, and runtime layers of LLM-agent systems.

5.1.1. System model

We consider a distributed multi-agent or federated environment:

$$\mathcal{E} = \{A_1, A_2, \dots, A_n\}, \quad (27)$$

where each agent A_i operates an internal model M_{θ_i} with parameters θ_i , a local dataset D_i , and participates in global coordination via a communication protocol $\mathcal{P}$. During normal operation, an agent computes:

$$y_i = M_{\theta_i}(x_i), \quad \text{and} \quad \theta_i^{t+1} = \text{Update}(\theta_i^t, \nabla_{\theta_i} \mathcal{L}_i), \quad (28)$$

where $\mathcal{L}_i$ is the local training or inference loss. Periodic synchronization occurs through aggregation functions (e.g., in federated learning):

$$\theta_G = \text{Agg}(\{\theta_i\}_{i=1}^n), \quad (29)$$

producing a global model θ_G shared among participants.

5.1.2. Adversary model

The adversary $\mathcal{A}$ may possess different privilege levels:

- *Local adversary*: Can access one or more D_i , gradients ∇_{θ_i} , or outputs y_i .
- *Network adversary*: Can observe timing or message metadata within $\mathcal{P}$.
- *Global adversary*: Can tamper with $\text{Agg}(\cdot)$ or the communication scheduling process.

The attack goal is to infer, reconstruct, or extract sensitive information from D_i or intermediate states while maintaining stealth. Formally:

$$\hat{x} = \mathcal{A}(y_i, \theta_i, \mathcal{P}) \quad \text{s.t.} \quad \text{Sim}(\hat{x}, x_i) \geq \tau, \quad (30)$$

where $\text{Sim}(\cdot)$ is a semantic or embedding similarity function and τ a privacy-breach threshold.

5.1.3. Extraction & privacy attacks

These attacks exploit correlations between model outputs and sensitive training samples. Formally, a membership inference or extraction attack tests the hypothesis:

$$H_0 : x \notin D_i \quad \text{vs.} \quad H_1 : x \in D_i, \quad (31)$$

by computing an inference score $s(x)$ derived from model confidence or reconstruction similarity. An attack is successful if:

$$\Pr[H_1 | s(x) \geq \lambda] \geq \eta, \quad (32)$$

where λ is a threshold and η the attacker's confidence level.

5.1.4. Federated & multi-agent attacks

In federated or collaborative settings, adversaries can corrupt local updates or inject crafted gradients. Formally, the adversary perturbs a local model as:

$$\theta'_i = \theta_i + \delta_i, \quad (33)$$

where δ_i is crafted to maximize global divergence:

$$\max_{\delta_i} D(\text{Agg}(\{\theta'_i\}), \theta_G), \quad (34)$$

subject to stealth constraints $\|\delta_i\| \leq \epsilon$. This captures poisoning and model backdoor attacks in decentralized systems.

5.1.5. Side-channel & timing attacks

Side-channel attacks exploit auxiliary information from timing, memory, or bandwidth patterns. Let $T(x)$ and $S(x)$ denote timing and size traces of the LLM service for input x . The adversary estimates a leakage function:

$$I_{\text{leak}} = \mathbb{E}_{x \in \mathcal{X}} [I(x; T(x), S(x))], \quad (35)$$

where $I(\cdot; \cdot)$ is mutual information. The goal is to maximize I_{leak} to reveal sensitive correlations between encrypted communication and model-internal states.

5.1.6. Relationship among attack classes

System & Privacy Attacks form a unified threat class $\mathcal{G}_{\text{sys}}$ composed of inference-based ($\mathcal{G}_{\text{inf}}$), corruption-based ($\mathcal{G}_{\text{fed}}$), and leakage-based ($\mathcal{G}_{\text{side}}$) submodels:

$$\mathcal{G}_{\text{sys}} = \mathcal{G}_{\text{inf}} \cup \mathcal{G}_{\text{fed}} \cup \mathcal{G}_{\text{side}}. \quad (36)$$

These attacks differ in vector type (data vs. gradient vs. signal) but converge in intent—to compromise the confidentiality and reliability of LLM-agent ecosystems operating across distributed or multi-agent environments.

5.2. Extraction & privacy attacks

5.2.1. Speculative decoding side-channel attack

Soleimani et al. [63] address a critical security vulnerability introduced by recent server-side optimizations, such as speculative decoding, in large language model (LLM) services. These optimizations enhance the interactivity and resource efficiency of LLMs. Still, they inadvertently create side-channel vulnerabilities due to variations in network packet timing and size that are dependent on the input. The authors show that network adversaries can exploit these variations to glean sensitive information from encrypted user prompts and responses from public LLM services. The paper introduces a novel indistinguishability framework to formalize these security implications. It presents a new attack method that deconstructs encrypted network packet traces to uncover details about LLM-generated tokens. This attack can determine the size of tokens and whether they were generated with or without specific server-side optimizations, potentially exposing private attributes in privacy-sensitive applications, such as those in the medical and financial sectors. Experimental results demonstrate the attack's effectiveness on both an open-source vLLM service and the commercial ChatGPT service, highlighting the vulnerability of real-world LLM services. Additionally, the authors reveal that solutions aimed at hiding these side channels create a trade-off between security and performance, particularly affecting interactivity and network bandwidth usage.

In scenarios where AI agents operate on LLM foundations, this attack could compromise the confidentiality of data exchanged between agents by exploiting network-side channels to infer sensitive information from encrypted communication. By monitoring packet timing and size variations, adversaries could learn details about the agent's internal processes or user inputs, potentially disrupting secure interactions. This underscores the need for enhanced security measures to safeguard against side-channel attacks in multi-agent systems, ensuring sensitive data remains protected even as performance and interactivity are optimized.

5.2.2. Membership attacks

Li et al. [39] address a critical yet underexplored privacy risk in Retrieval-Augmented Generation (RAG) systems, specifically focusing on membership privacy concerns related to the external databases integrated into these models. While RAG enhances Large Language Models (LLMs) by retrieving relevant knowledge from an external database to mitigate hallucinations and knowledge staleness, incorporating sensitive data — such as medical records or personal identities

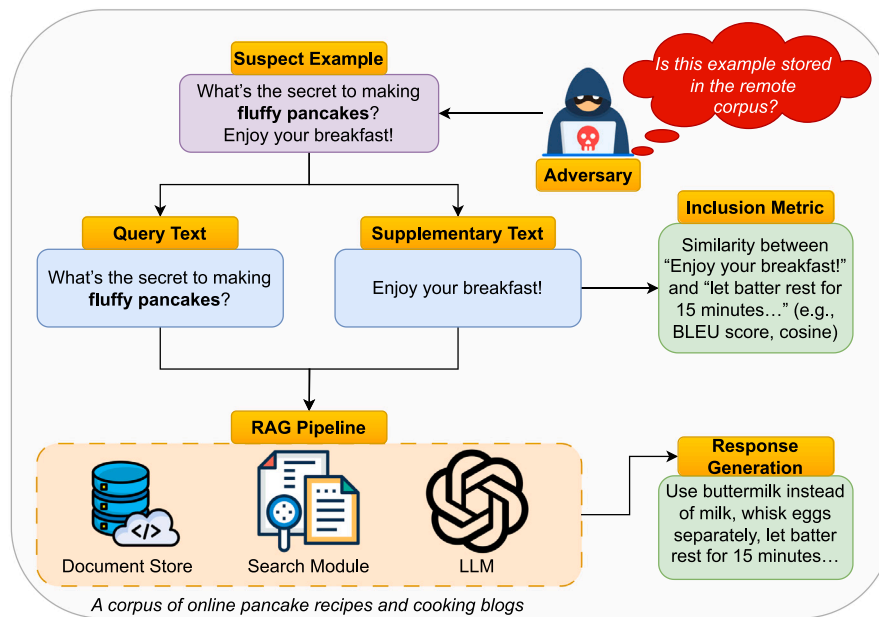


Fig. 13. Membership inference attack.

— raises significant privacy concerns. The authors introduce S2MIA, a Membership Inference Attack that exploits semantic similarity between a given sample and the RAG system's generated content to determine whether the sample is in the RAG's database. The key idea is that samples in the external database exhibit greater semantic alignment with the generated text, allowing them to be identified through this similarity. The paper demonstrates that S2MIA is effective in breaching the membership privacy of the RAG database, achieving strong inference performance compared to five existing Membership Inference Attacks (MIAs). Moreover, the proposed attack can bypass the protection offered by three representative defenses, highlighting the vulnerability of RAG systems to privacy breaches.

Fig. 13 shows the S2MIA approach applied to a cooking scenario. An adversary splits the user's input — "What's the secret to making fluffy pancakes? Enjoy your breakfast!" — into a **query segment** ("What's the secret to making fluffy pancakes?") and a **supplementary text** ("Enjoy your breakfast!"). The query is sent through a Retrieval-Augmented Generation pipeline that fetches relevant pancake recipes from an external document store and conditions the language model to generate a cooking tip (e.g., "Use buttermilk instead of milk, whisk eggs separately, let batter rest for 15 min..."). Finally, an **inclusion metric** (such as BLEU or cosine similarity) is computed between the generated tip and the supplementary text. A high similarity score reveals that the original pancake prompt was present in the recipe database, enabling the adversary to infer membership.

In the context of LLM-based AI agents, S2MIA could be used to exploit the external knowledge databases these agents rely on, potentially exposing private data embedded in them. By targeting the membership privacy of these external databases, attackers could infer which specific data points are being accessed or used in the agent's responses, compromising user confidentiality. This underscores the importance of developing robust privacy protections for external knowledge sources integrated with LLM-based agents, ensuring that sensitive information remains secure and that agents do not inadvertently expose confidential data during user interaction.

5.3. Federated & multi-agent system attacks

5.3.1. Contagious recursive blocking attacks

Zhou et al. [64] introduce Contagious Recursive Blocking Attacks (Corba), a novel and highly effective attack method targeting Large

Language Model-based Multi-Agent Systems (LLM-MASs). While LLM-MASs have demonstrated impressive capabilities in real-world applications, collaborating to complete complex tasks, their security remains largely underexplored. These systems are designed with safety mechanisms, such as alignment checks to reject harmful instructions, but they remain vulnerable to targeted disruptions. Corba exploits two key properties: its contagious nature, which allows the attack to propagate across various network topologies, and its recursive property, which depletes computational resources over time. The attack typically involves seemingly benign instructions, making it difficult to detect and mitigate using traditional alignment methods. The authors evaluate Corba on two widely used LLM-MASs, AutoGen and Camel, across various topologies and commercial models. Through extensive experiments in both controlled and open-ended interactive environments, they demonstrate the effectiveness of Corba in disrupting agent interactions within complex topology structures and open-source models. These findings underscore the need for robust security measures to safeguard LLM-MASs against disruptive attacks, particularly those that employ subtle and seemingly harmless instructions.

5.3.2. Federated LLM attack

Han et al. [61] introduce FedSecurity, an end-to-end benchmark designed as a supplementary component of the FedML library to simulate adversarial attacks and defense mechanisms in Federated Learning (FL). FedSecurity simplifies the implementation of fundamental FL procedures, such as training and data loading, allowing users to focus on developing and testing their own attack and defense strategies. The benchmark consists of two key components: FedAttacker, which executes various adversarial attacks during FL training, and FedDefender, which implements defensive mechanisms to mitigate these attacks. FedSecurity offers extensive customization options, making it compatible with a wide range of machine learning models, including Logistic Regression, ResNet, and GANs, as well as FL optimizers such as FedAVG, FedOPT, and FedNOVA. The framework enables users to explore the effectiveness of various attacks and defenses across different datasets and models, and it supports flexible configuration via a configuration file and APIs. To demonstrate its utility and adaptability, the authors showcase FedSecurity's application in federated training of Large Language Models (LLMs), highlighting its potential to address complex applications in FL.

Table 6

Summary of Attacker Privilege, Feasibility, and Layer Impact Across Major Attack Categories in LLM-Agent Ecosystems. **Typical Privilege:** Minimum access level required by the adversary (e.g., none, user, or system). **Feasibility:** Practical likelihood of executing or reproducing the attack in real-world scenarios (Low, Medium, or High). **Primary Layer Impacted:** Key subsystem affected within the LLM-agent architecture, encompassing the Host, Tool, Agent, or Protocol layer..

Attack category	Typical privilege	Feasibility (Low/Medium/High)	Primary layer impacted
Input Manipulation Attacks (e.g., Prompt Injection, Jailbreaking, Context Hijacking)	None to Low (user-level or public API access)	High — easily reproducible through open LLM endpoints and exposed toolchains	Tool/Agent
Model Compromise Attacks (e.g., Backdoor, Data Poisoning, Memory Injection)	Medium to High (training, fine-tuning, or data pipeline access)	Medium — requires control over model training or update workflow	Host/Model
System & Privacy Attacks (e.g., Membership Inference, Retrieval Poisoning, Side Channels)	Medium (runtime or federated environment access)	Medium — feasible under shared or multi-agent execution contexts	Host/Agent
Protocol Vulnerabilities (e.g., MCP, ACP, ANP, A2A Exploits)	Low to Medium (connector, API, or communication access)	High — affects integration boundaries and message exchanges between agents	Protocol/Agent

In the scope of AI agents leveraging LLM architectures, FedSecurity can simulate adversarial attacks and defense mechanisms within federated settings, ensuring that AI agents are trained securely and robustly. Using FedAttacker, researchers can evaluate how well multi-agent systems perform under adversarial conditions, while FedDefender can help optimize defense strategies to safeguard agent communications. This benchmark helps enhance the security of decentralized AI systems, ensuring agents can interact safely while maintaining performance even under malicious attacks.

5.4. Social-engineering & data-extraction attacks

5.4.1. Datastore leakage attack

Qi et al. [62] investigate the risk of datastore leakage in Retrieval-Augmented Generation (RAG) Language Models (LMs), focusing on the vulnerability that allows adversaries to extract verbatim text data from the datastore of RAG systems. By leveraging LMs' instruction-following capabilities, the authors demonstrate that adversaries can easily exploit prompt injection to extract sensitive information from instruction-tuned models, including Llama2, Mistral/Mixtral, Vicuna, SOLAR, WizardLM, Qwen1.5, and Platypus2. The exploitability of this vulnerability increases as model size scales up, making larger models more susceptible. The paper also explores how RAG setup influences data extractability, revealing that failing to effectively utilize context in modern LMs can lead to unexpected instructions that cause the model to regurgitate sensitive data. The authors propose that position bias elimination strategies can significantly mitigate this vulnerability. Extending their research to production RAG models, the authors demonstrate an attack that achieves a 100% success rate on 25 randomly selected customized GPT models, extracting text data verbatim from a 77,000-word book at a rate of 41%, and a 1,569,000-word corpus at a rate of 3%, using only 100 queries generated by the models themselves.

Regarding AI agents built on LLM platforms, this datastore leakage vulnerability could be exploited to extract sensitive information from the agents' memory banks, thereby compromising the confidentiality of the data they use. By manipulating input prompts, adversaries could trigger agents to reveal stored knowledge, including private or proprietary information, without needing direct access to the underlying datastore. This highlights the need for robust security measures, such as advanced context management and query filtering, to prevent LLM agents from inadvertently exposing sensitive information during interactions.

5.4.2. Social engineering attack

Kumarage et al. [65] introduce a novel LLM-agentic framework, SE-VSim, designed to simulate social engineering (SE) attacks in multi-turn, chat-based interactions, particularly in the context of conversational agents, such as chatbots powered by large language models

(LLMs). The authors highlight that SE detection in multi-turn conversations is significantly more complex than in single-instance interactions, due to the dynamic, evolving nature of these conversations. A key challenge in mitigating SE attacks is understanding the mechanisms through which attackers exploit vulnerabilities, as well as how victims' personality traits influence their susceptibility to manipulation. To explore this, SE-VSim models victim agents with varying psychological profiles to assess the influence of personality traits on vulnerability to SE attacks. Using a dataset of over 1000 simulated conversations, the authors simulate attack scenarios where adversaries, posing as recruiters, funding agencies, and journalists, attempt to extract sensitive information. The paper further presents a proof-of-concept, SE-OmniGuard, a personalized protection mechanism that leverages prior knowledge of the victim's personality, evaluates attack strategies, and monitors information exchanges during conversations to detect and prevent potential SE attempts.

For AI agents equipped with large language model capabilities, this framework can simulate and analyze their responses to SE attacks, enabling the identification of vulnerabilities in communication protocols. By understanding how different personality traits influence susceptibility, defensive strategies like SE-OmniGuard could be developed to enhance the resilience of multi-agent systems, ensuring that AI agents can protect sensitive information while interacting with users. This highlights the need for incorporating psychological profiling into the design of secure and trustworthy AI agents, particularly in domains where agents handle sensitive or confidential interactions.

5.5. Attacker privileges, feasibility, and layer impact analysis

While the preceding subsections summarized individual research efforts across multiple attack domains, it is equally important to contextualize how these attacks differ in terms of adversarial capability, real-world feasibility, and the specific system layers they impact. This subsection introduces a cross-domain analysis that links the four major categories in our taxonomy — Input Manipulation, Model Compromise, System and Privacy Attacks, and Protocol Vulnerabilities — to their operational requirements and affected components within the LLM-agent ecosystem.

From a privilege perspective, most *input manipulation attacks* (e.g., prompt injections [40,41], adaptive indirect injections [20], or compositional jailbreaks [21]) require no special access to the model or infrastructure. These are typically carried out through public interfaces, API endpoints, or interactive tools exposed to end users. Conversely, *model compromise attacks* (e.g., backdoors [25,26], data poisoning [27, 56]) demand moderate to high privileges, as they exploit training data pipelines or internal model weights. In contrast, *system and privacy*

attacks occupy a middle ground: they often target runtime memory, retrieval pipelines, or federated model states—areas accessible only to semi-trusted agents or insiders [39,63]. Finally, *protocol vulnerabilities* such as those affecting MCP, ACP, ANP, or A2A involve low to moderate privileges but exhibit high exploitability, as shown in real-world cases such as the Toxic Agent Flow attack or recursive blocking exploits [64].

Feasibility is similarly uneven across categories. Input manipulation and protocol-level exploits are highly feasible [22,23], given their reproducibility via public LLM endpoints and the widespread use of loosely validated APIs. Model compromise and system-level attacks, although more sophisticated, are increasingly practical in shared or open-source environments where fine-tuning, memory access, or federated participation is exposed [60,61]. This asymmetry underscores the layered risk landscape: while high-impact attacks may require privileged access, low-privilege attacks remain far more scalable and frequent. Therefore, the affected layers vary as well. Input manipulation attacks primarily compromise the *tool* and *agent* layers, influencing behavior through external inputs. Model compromise targets the *host* or *model* layers, manipulating training dynamics or parameter space. System and privacy attacks extend into the *host* and *agent* layers, exploiting runtime environments or shared retrieval caches. Finally, protocol vulnerabilities operate at the *protocol-agent* boundary, where data, authentication, or control signals are exchanged between agents and tools [17,31].

Table 6 summarizes these contextual dimensions — privilege level, feasibility, and affected layers — providing a practical mapping that complements the detailed literature review. This structured view supports threat modeling, helps prioritize mitigations based on attacker capabilities, and identifies which parts of the LLM-agent stack require urgent security reinforcement. As shown in Table 6, attacks that require minimal privileges — such as prompt injections or protocol exploits — remain the most prevalent and replicable in practice, posing a disproportionate security threat [20,40]. In contrast, deeper model-compromise or data-poisoning attacks, while harder to execute, carry severe long-term consequences by undermining the trustworthiness and alignment of the underlying LLM [25,26]. This analysis reinforces the need for multi-layered defense strategies that span access control, input validation, model integrity verification, and secure protocol design [31, 34].

6. Vulnerabilities & attacks for MCP and A2A protocols-based agent communications

Autonomous agents rely on standardized communication protocols — most notably MCP and A2A — to extend language models with external tools and coordinate multi-agent workflows. In this work, the term *protocol vulnerabilities* refers specifically to weaknesses in agent coordination and communication standards rather than general network protocols. These include the Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent Network Protocol (ANP), and Agent-to-Agent (A2A) protocol, which govern structured interactions between LLM-powered agents and external tools or services. Such vulnerabilities arise from misconfigured authentication, context manipulation, and insecure orchestration flows, distinguishing them from traditional transport-layer issues (e.g., HTTP or gRPC exploits).

While these protocols greatly enhance flexibility and interoperability, they also introduce rich new attack surfaces at both the prompt-processing and orchestration layers. In the subsections that follow, we discuss the primary vulnerability classes, ranging from prompt-based injections and backdoor mechanisms to discovery spoofing and recursive denial-of-service, and illustrate how adversaries can subvert protocol semantics, manipulate context, or abuse authentication to compromise agent communications.

6.1. Formal definition of the protocol-level threat model

To rigorously capture the semantics of communication-layer vulnerabilities among autonomous agents, we formalize the *Protocol-Level Threat Model*, which encompasses the Model Context Protocol (MCP), Agent Network Protocol (ANP), Agent Communication Protocol (ACP), and Agent-to-Agent (A2A) interactions. These protocols introduce structured message exchanges between language-model-driven agents and their tools, databases, or peers. Vulnerabilities at this level arise not from model weights but from insecure message orchestration, authentication misconfigurations, or context propagation flaws across agents. The following formulation defines how such attacks can be represented mathematically and classified across the orchestration hierarchy.

6.1.1. System model

Let $\mathcal{A} = \{A_1, A_2, \dots, A_n\}$ denote a set of autonomous agents, each equipped with an internal language model M_{θ_i} and a communication interface defined by a protocol $P_i = (\mathcal{M}_i, \mathcal{K}_i, \mathcal{R}_i)$, where:

- $\mathcal{M}_i$ is the message schema or API specification (e.g., MCP request/response, JSON-RPC task object);
- $\mathcal{K}_i$ is the context store (memory, embeddings, or retrieved results);
- $\mathcal{R}_i$ is the rule set governing message sequencing, authentication, and verification.

Each agent transmits or receives structured messages:

$$m_{ij} = \langle \text{id}, \text{src}, \text{dst}, \text{type}, \text{payload}, \sigma \rangle, \quad (37)$$

where σ is the authentication or integrity signature. A valid exchange satisfies:

$$\text{Verify}_{\mathcal{R}_j}(m_{ij}, \sigma) = 1. \quad (38)$$

The agent output at time t is thus a function of the model state, received messages, and retrieved context:

$$y'_i = M_{\theta_i}(x'_i, \mathcal{K}_i, \mathcal{M}_i). \quad (39)$$

6.1.2. Adversary model

An adversary $\mathcal{A}^*$ can manipulate or intercept protocol messages, impersonate an agent, or alter context values. Formally, the adversary introduces a transformation g_p over the message space:

$$g_p : m_{ij} \mapsto m'_{ij} = \langle \text{id}', \text{src}', \text{dst}', \text{type}', \text{payload}', \sigma' \rangle, \quad (40)$$

subject to one or more of the following goals:

$$\text{Integrityviolation} : m'_{ij} \neq m_{ij} \wedge \text{Verify}_{\mathcal{R}_j}(m'_{ij}, \sigma') = 1, \quad (41)$$

$$\text{Impersonation} : \text{src}' \neq \text{src}, \quad (42)$$

$$\text{Contexttampering} : \mathcal{K}'_i = \mathcal{K}_i \oplus \delta_k, \quad (43)$$

$$\text{Denialofservice} : \exists t : |\{m'_{ij}\}| > \tau_{\text{rate}}, \quad (44)$$

where τ_{rate} is the resource threshold. The attack succeeds if the protocol-level policy $\pi_{\text{proto}}(m'_{ij}) = 0$, i.e., the modified exchange violates communication or access control constraints.

6.1.3. Authentication & credential attacks

Authentication and token-theft attacks can be expressed as violations of signature consistency:

$$\sigma' \neq \text{Sign}_{\text{auth}}(\text{src}', \text{payload}') \text{ and } \text{Verify}_{\mathcal{R}_j}(m'_{ij}, \sigma') = 1. \quad (45)$$

This occurs when replayed, stolen, or confused-deputy tokens remain cryptographically valid due to misconfigured verification or missing nonce enforcement in $\mathcal{R}_j$.

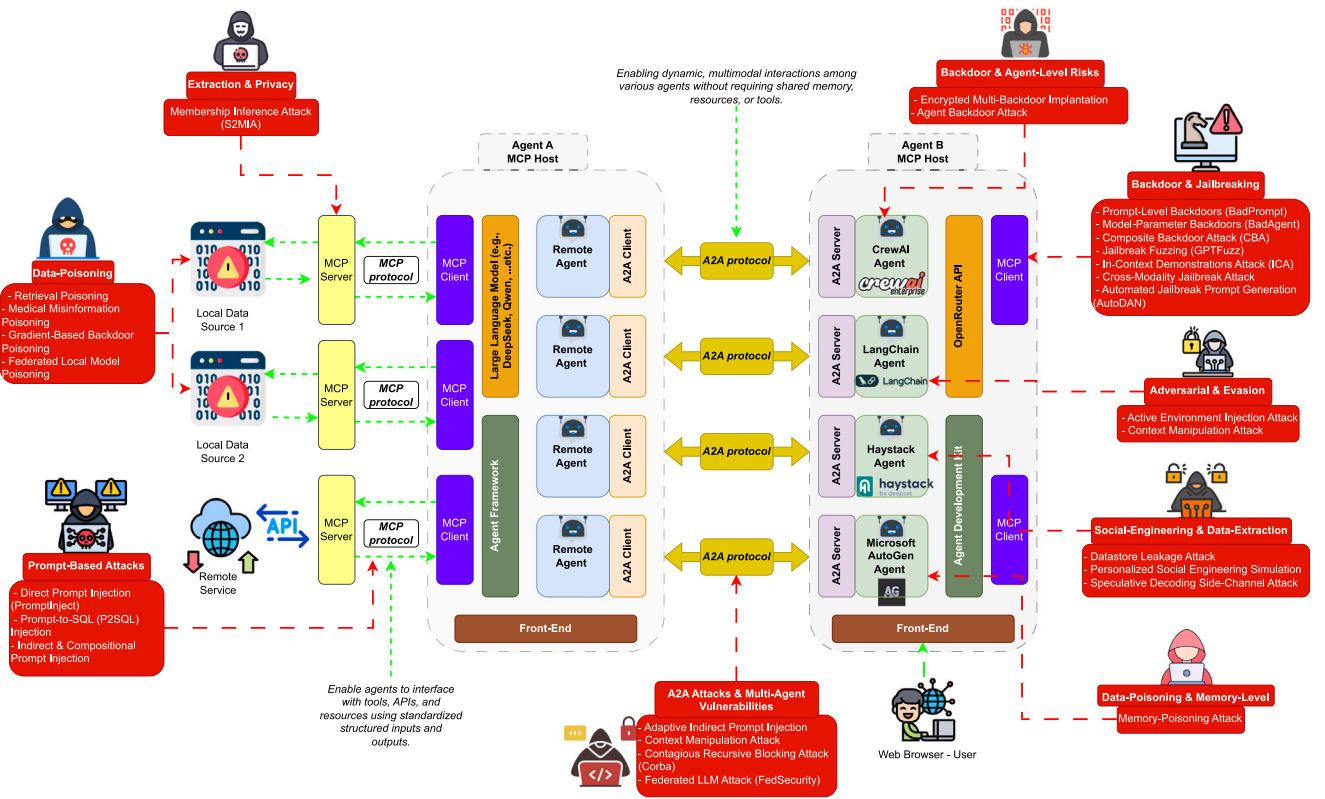


Fig. 14. Vulnerabilities and attack techniques in LLM-powered AI agent communications.

6.1.4. Cross-agent context manipulation

Context manipulation attacks exploit unvalidated state propagation between agents. Given a context aggregation function:

$$\mathcal{K}_i^{t+1} = f(\mathcal{K}_i^t, \text{payload}(m_{ji})), \quad (46)$$

the adversary injects δ_k such that:

$$\mathcal{K}_i^{t+1} = f(\mathcal{K}_i^t \oplus \delta_k, \text{payload}(m'_{ji})), \quad (47)$$

resulting in erroneous or malicious context influencing downstream decisions. This mechanism underlies cross-agent prompt injection, discovery spoofing, and replay-based corruption.

6.1.5. Recursive & contagious workflow exploits

Recursive DoS and contagious blocking occur when agents recursively generate tasks:

$$T_i^{t+1} = g(T_i^t) = \text{Spawn}(A_j, T_i^t), \quad (48)$$

creating unbounded task trees $|T| \rightarrow \infty$. If resource consumption exceeds τ_{cap} , the orchestrator halts or crashes, leading to distributed denial of service across the A2A network.

6.1.6. Protocol-level threat composition

The complete protocol threat model can be expressed as:

$$\mathcal{G}_{\text{proto}} = \mathcal{G}_{\text{auth}} \cup \mathcal{G}_{\text{ctx}} \cup \mathcal{G}_{\text{dos}}, \quad (49)$$

representing the union of authentication/credential attacks ($\mathcal{G}_{\text{auth}}$), context-manipulation exploits ($\mathcal{G}_{\text{ctx}}$), and recursive/DoS vectors ($\mathcal{G}_{\text{dos}}$). Each submodel corresponds to a distinct layer of the agent-communication pipeline, mapping MCP to vertical integration (tool invocation) and A2A to horizontal collaboration (multi-agent orchestration). Together, they define the uppermost layer of the end-to-end LLM-Agent Security Stack.

6.2. Model-control-protocol (MCP)

MCP [66] and A2A [67] are complementary in enabling robust communication among LLM-powered agents. MCP (Model Context Protocol) is a standardized bridge between a language model and the external resources it requires during inference. By defining a client-server architecture in which MCP clients embedded within agent hosts request specific capabilities — such as file retrieval, database queries, or external API calls — from lightweight MCP servers, agents can dynamically augment their context window without hardcoding tool integrations. In this way, MCP enables vertical integration, allowing an LLM to invoke structured tools and access relevant information (e.g., fetching a document or executing a function) in real time, thereby enhancing the agent's reasoning with up-to-date, application-specific data.

The MCP component of the architecture serves as a unified interface between each agent and its underlying data sources or tool integrations. In practice, an MCP host (e.g., an IDE or desktop application) instantiates a lightweight MCP client that maintains a one-to-one connection with corresponding MCP servers. These servers expose discrete capabilities, such as database access, document retrieval, or specialized API calls, through a standardized protocol. By encapsulating complex data-access logic behind the MCP layer, agents can dynamically request contextual information or invoke external services without needing bespoke connectors for each resource. In the illustrated framework, each local data repository is coupled to an MCP server that performs authentication, enforces access policies, and mediates queries. Downstream, the collocated MCP client within each agent host transforms those raw data streams into structured prompts or tool invocations, ensuring that every interaction adheres to a consistent security posture and format. Decoupling data access from agent logic reduces engineering overhead and enables agents to interchange MCP-compatible resources, facilitating seamless vendor-agnostic collaboration with minimal reconfiguration.

Table 7
Comprehensive security threat matrix for MCP [66], A2A [67], ANP [68] & ACP [69] Protocols.

Attack	Category	Protocol(s)	Impact	Mitigation
Prompt Injection	Injection	MCP, A2A, ANP, ACP	Unintended commands; data exfiltration	Input sanitization; strict prompt/schema validation
Supply Chain & Metadata Poisoning	Poisoning	MCP, A2A, ANP, ACP	Toolchain compromise; downstream infection	Code signing; integrity monitoring
Token Theft & Replay	Authentication	MCP, A2A, ANP, ACP	Agent takeover; impersonation	Short-lived tokens; nonce/timestamp checks; mTLS
Man-in-the-Middle (MitM)	Interception	MCP, A2A, ANP, ACP	Data theft; tampering	Mutual TLS; end-to-end encryption
Replay Attack	Replay	MCP, A2A, ANP, ACP	Duplicate/unauthorized actions	Nonce/timestamp enforcement; unique message IDs
Denial-of-Service (DoS)	Resource Exhaustion	MCP, A2A, ANP, ACP	Service unavailability	Rate limiting; circuit breakers
Insecure Configuration	Misconfiguration	MCP, A2A, ANP, ACP	Unauthorized access	Secure defaults; regular audits; least-privilege settings
Agent Discovery Spoofing	Spoofing	A2A, ANP	Workflow hijacking	Signed discovery responses; authenticated peer discovery
Rogue Agent Registration	Unauthorized Registration	A2A, ANP	Malicious task execution	Agent whitelisting; strict onboarding policies
Peer Discovery Poisoning	Poisoning	ANP	Malicious peer injection	Trust anchors; mutual authentication of peers
Schema Poisoning	Poisoning	ACP	Validation bypass; malicious schema execution	Schema signing; strict JSON-RPC schema enforcement
Capability Escalation	Authorization Abuse	ANP, ACP	Privilege escalation	Fine-grained ACLs; runtime authorization checks
Credential theft	Proxy Abuse	MCP, ANP, ACP	Unauthorized API calls under elevated identity	Token audience binding; explicit delegation checks
Cross-Agent Prompt Injection	Injection	A2A, ANP	Cascading malicious actions across agents	Input isolation; per-agent validation

```
// MCP TypeScript SDK invocation with injection point
import { McpClient } from "@modelcontextprotocol/sdk/client/mcp.js";
import { StreamableHttpTransport } from "@modelcontextprotocol/sdk/client/http.js";

async function runQuery(userInput: string) {
  const client = new McpClient({
    transport: new StreamableHttpTransport("https://mcp.example.com"),
  });
  const response = await client.invoke({
    tool: "database_query",
    args: {
      // <-- Injection point
      query: userInput
    }
  });
  console.log(response.content);
}

// Benign call
runQuery("SELECT * FROM users WHERE id=123");

// Malicious call
runQuery("SELECT * FROM users WHERE id=123; DROP TABLE users; --");
```

Fig. 15. MCP TypeScript SDK invocation showing where a SQL injection payload can slip in.

MCP-layer Attack Vectors. Despite these benefits, MCP introduces a range of new attack surfaces, as presented in Fig. 14. Key vectors include:

- **Prompt-based injections:** direct prompt injection, prompt-to-SQL injection, and indirect/compositional injections that lead to unauthorized tool calls. These attacks manipulate the natural-language prompts sent to the MCP server, causing unintended or malicious tool invocations that compromise data integrity or confidentiality. Fig. 15 highlights a critical vulnerability in the MCP TypeScript SDK where unvalidated user input is passed directly into the ‘args.query’ field and subsequently executed by the database. Because the SDK serializes the ‘userInput’ string into the JSON payload, an attacker can embed additional SQL commands (e.g., ‘DROP TABLE users;’) that the backend will execute without sanitation.
- **Backdoor mechanisms:** prompt-level, model-parameter, and composite backdoors that trigger malicious behavior. By embedding hidden triggers within prompts or model weights, attackers can activate payloads that subvert the agent’s logic or bypass safety checks.
- **Fuzzing attacks:** jailbreak fuzzing and automated jailbreak-prompt generation to discover escape sequences. Automated tools generate variations of input sequences to probe for vulnerabilities in the MCP handling logic, enabling large-scale discovery of jailbreak conditions.

- *Data poisoning*: retrieval poisoning, medical-misinformation poisoning, gradient-based backdoor poisoning, federated model poisoning. Compromised training or retrieval datasets introduce malicious or misleading information, undermining downstream reasoning and decisions.
- *Privacy-extraction*: membership inference and side-channel (speculative decoding) attacks. Adversaries infer sensitive data or model internals by observing query patterns or timing characteristics of MCP interactions.
- *Replay and DoS*: replay attacks and denial-of-service via infinite tool-call loops or request floods. Captured requests can be resent to duplicate actions, while resource exhaustion attacks degrade the availability of MCP services.
- *Credential theft*: token theft/replay, confused-deputy proxy abuse, and insecure configuration. Attackers steal or misuse authentication tokens to impersonate agents and hijack privileged MCP operations.

Table 8 presents representative CVEs from 2025 cross-mapped to the four proposed threat model categories, demonstrating the empirical grounding and real-world relevance of our taxonomy. These vulnerabilities span indirect prompt injection (e.g., LangChain’s GmailToolkit), server-side request forgery, XML-based data exfiltration, path traversal, protocol misconfigurations (e.g., in MCP-Remote), and sandbox escapes in agent tools. Notably, most of these attacks exploit insecure interfaces, permissive plugin logic, or protocol-level flaws — often requiring only low privileges or indirect input access — highlighting the brittleness of current LLM-agent integrations. This validates the practical urgency of securing prompt surfaces, agent-tool bridges, and multi-agent coordination flows through structured, layered defenses.

6.3. Agent2Agent (A2A) protocol

A2A (Agent-to-Agent) [67] extends this framework horizontally by orchestrating secure, task-oriented interactions among multiple autonomous agents. Leveraging a JSON-RPC-over-HTTP(S) communication model with Server-Sent Events for streaming updates, A2A enables one agent (the client) to discover and delegate subtasks to another agent (the remote) based on metadata published in “Agent Cards”. Each task is a defined object whose lifecycle — from initiation to artifact delivery — is governed by the protocol, allowing agents to coordinate complex workflows and negotiate content formats (text, images, forms, etc.) without sharing internal memory or tools. When combined, MCP equips each agent with the precise context it needs, while A2A orchestrates multi-agent collaboration, supporting scalable, interoperable ecosystems of LLM-enabled agents [70].

Through A2A’s capability discovery mechanism, each agent advertises its skills in a JSON “Agent Card”, allowing a client agent to identify and negotiate with the most appropriate remote agent for a given task. Once a suitable match is found, A2A orchestrates the task lifecycle: the client agent transmits a formally defined “task” object, and the remote agent executes the required operations or returns an artifact. Throughout execution — whether a quick database lookup or a multi-day research workflow — A2A ensures secure, authenticated message exchanges via HTTP, Server-Sent Events, and JSON-RPC conventions. Modality-agnostic abstractions within A2A enable seamless transfer of text, images, or video, and real-time state updates maintain continuity during long-running tasks.

A2A-layer Attack Vectors. In addition to the MCP-layer threats above, A2A protocols are vulnerable to:

- *Cross-agent prompt injection*: crafted prompts that propagate through multi-agent workflows. These attacks embed malicious instructions in one agent’s output, causing downstream agents to perform unintended actions.

- *Agent discovery spoofing*: forged discovery responses or Agent-Card spoofing to impersonate malicious agents. The internal state leads to capability metadata to divert tasks to rogue agents under their control.
- *Rogue agent registration*: unauthorized or look-alike agents hijacking tasks. Malicious agents register with deceptive identifiers to intercept or tamper with delegated workflows.
- *Adaptive indirect injections*: dynamic chaining of prompts to evade filters. By incrementally modifying prompts across hops, attackers bypass individual-agent sanitization mechanisms.
- *Context manipulation*: injecting or modifying context blobs to hijack workflows. Altered context payloads lead agents to make incorrect decisions or reveal sensitive data.
- *Contagious recursive blocking*: recursive task spawning deadlocks creating denial-of-service. Agents inadvertently generate infinite loops of subtask delegation, exhausting system resources.
- *Memory subversion*: memory-poisoning (MINJA) within the agent framework. Injected memory artifacts corrupt the internal state, leading to unpredictable or malicious agent behavior.

Table 7 summarizes the attack vectors across MCP [66], A2A [67], ANP [68] & ACP [69] protocols.

7. Open challenges and future directions

As LLMs and agentic systems become increasingly pervasive, securing them against sophisticated threats remains a significant challenge. This section identifies the key areas where current defenses fall short and outlines promising directions for future research.

7.1. MCP protocol vulnerabilities

The Model Context Protocol (MCP) provides a unified, plug-and-play framework—akin to a “USB-C port for AI” for connecting large language models to diverse data sources and tool endpoints. By decoupling responsibilities into hosts (applications), clients (connectors), and servers (modular adapters), MCP greatly accelerates the assembly of complex, LLM-driven workflows. Yet this very modularity widens the attack surface: every integration point — from prompt handling to long-running connections — can be exploited if left unchecked [91].

The first significant challenge lies in establishing dynamic trust and adaptive policy enforcement within MCP deployments. Traditional static access controls cannot keep pace with the fluid, language-driven workflows that modern agents require. Future research should explore declarative policy languages that allow hosts to negotiate and update least-privilege rules at runtime, as well as distributed trust-negotiation protocols capable of granting time-boxed permissions for specific tool invocations without human intervention [31].

A second frontier concerns the integrity of context and provenance tracking. Because MCP maintains conversational state across multiple turns and service calls, any undetected tampering can subtly bias downstream actions. Promising directions include using secure enclaves or hardware root-of-trust modules to vault critical session memory and immutable audit logs — perhaps built on lightweight blockchain ledgers — to attest to the sequence of context updates cryptographically [92].

Ransomware [93] can cripple an MCP deployment by encrypting or locking access to the very data sources and tool-endpoints that MCP servers expose, effectively severing the “USB-C port” connections between LLMs and critical context (e.g., local files, databases, or APIs). Because MCP’s client-server architecture allows a host to plug into multiple lightweight servers, a single compromised server can cascade, denying clients the ability to fetch prompts, workflows, or sampling configurations and halting AI-driven operations until a ransom is paid, directly undermining MCP’s goals of flexible, secure integration within your infrastructure.

Table 8

Representative CVEs cross-mapped to the four threat model categories. CWE-94 (Code Injection), CWE-918 (Server-Side Request Forgery), CWE-200 (Information Exposure), CWE-611 (Improper Restriction of XML External Entity Reference), CWE-22 (Path Traversal), CWE-643 (XPath Injection), CWE-20 (Improper Input Validation), CWE-78 (OS Command Injection), and CWE-306 (Missing Authentication for Critical Function).

CVE ID	Date	Component affected	Summary of the issue	CWE-ID	Threat category
CVE-2025-46059 [71]	2025	LangChain – GmailToolkit (v0.3.51)	Critical indirect prompt injection via crafted emails enabling arbitrary code execution.	CWE-94 [72]	Input Manipulation
CVE-2025-2828 [73]	2025	LangChain – RequestsToolkit (OpenAPI tool)	SSRF flaw bypasses URL filters, allowing internal network access.	CWE-918 [74]	Protocol Vulnerabilities
CVE-2025-6984 [75]	2025	LangChain – EverNoteLoader (v0.3.63)	XML External Entity (XXE) attacks via insecure XML parsing expose sensitive files.	CWE-200 [76]	System & Privacy Attacks
CVE-2025-6985 [77]	2025	LangChain – HTMLSectionSplitter (v0.3.8)	Unsafe XSLT parsing allows file disclosure via malicious stylesheets.	CWE-611 [78]	System & Privacy Attacks
CVE-2025-6853 [79]	2025	LangChain-Chatchat (v0.3.1) – Backend API	Path traversal in temp upload allows out-of-scope file creation.	CWE-22 [80]	System & Privacy Attacks
CVE-2025-6854 [81]	2025	LangChain-Chatchat (v0.3.1) – Backend API	Base64 path traversal lets an attacker read unintended files.	CWE-22 [80]	System & Privacy Attacks
CVE-2025-6855 [82]	2025	LangChain-Chatchat (v0.3.1) – Backend API	Path traversal in file uploads allows overwriting critical files.	CWE-22 [80]	System & Privacy Attacks
CVE-2025-11844 [83]	2025	Hugging Face – SmolAgents (v1.20.0)	XPath injection lets attacker bypass filters, access unauthorized DOM elements.	CWE-643 [84]	Input Manipulation
CVE-2025-59532 [85]	2025	OpenAI – Codex CLI (v0.2.0-0.38.0)	Sandbox escape via directory control flaw allows arbitrary file execution.	CWE-20 [86]	Model Compromise
CVE-2025-6514 [87]	2025	Anthropic – MCP-Remote (v0.0.5-0.1.15)	Untrusted MCP endpoints can trigger remote OS command execution.	CWE-78 [88]	Protocol Vulnerabilities
CVE-2025-49596 [89]	2025	Anthropic – MCP Inspector (v0.14.0)	CSRF/DNS-rebinding vulnerability enables RCE via Inspector UI.	CWE-306 [90]	Protocol Vulnerabilities

Finally, there is an urgent need for anomaly detection and formal verification tailored to MCP's unique attack surface. Standard intrusion-detection systems are poorly suited to natural-language payloads; instead, machine-learning models must be trained to recognize semantic irregularities in prompts, tool responses, and context transitions. At the same time, formally specifying MCP's client-server interactions and verifying them against security properties can prevent entire logic-flaw vulnerabilities before deployment. By integrating zero-trust principles, cryptographic attestation of tool binaries, context-aware anomaly detection, and rigorous formal methods, the next generation of MCP implementations can combine the agility of modular AI workflows with the robust security guarantees required for production use.

7.2. Agentic web interfaces: Design and security challenges

Lu et al. [94] present a forward-thinking perspective on advancing web-based AI agents by proposing the Agentic Web Interface (AWI), a new interaction paradigm tailored to the unique capabilities and limitations of LLM-powered agents. As LLMs and their multimodal counterparts increasingly demonstrate the potential to automate complex web tasks, the fundamental misalignment between human-centric web interfaces and agent requirements has become a critical bottleneck. Current approaches, which attempt to adapt agents to navigate intricate web inputs such as dense Document Object Model (DOM) trees, screenshot-based environments, or direct API interactions, have proven insufficient in addressing these challenges. The authors introduce AWI, a standardized, agent-optimized interface that enables more efficient, reliable, and transparent interactions between AI agents and web environments, thereby overcoming these limitations. They outline six core design principles for AWI, focusing on safety, efficiency, and standardization, and emphasize the importance of collaboration across the machine learning community to realize this vision.

Despite its promise, the introduction of AWI also raises several open problems and security-related challenges that must be addressed to ensure safe deployment. First, introducing agent-optimized interfaces may inadvertently create new vulnerabilities, including unauthorized agent access, manipulation of AWI components, or exploitation of poorly implemented standardization protocols. Furthermore, as AWI enables more autonomous agent behaviors, attackers may exploit LLM-driven agents for malicious activities such as automated phishing, data exfiltration, or denial-of-service attacks within web environments. Future research should focus on developing robust security mechanisms

for AWI, including authentication, integrity verification, and real-time anomaly detection, to effectively monitor agent behavior. Additionally, establishing secure communication standards and incorporating adversarial robustness testing for LLM-agent interactions will be essential. Promising research directions include exploring formal verification techniques for AWI protocols, creating secure sandbox environments for agent testing, and developing mechanisms for transparent auditing of agent actions. Addressing these vulnerabilities is crucial to ensure that the shift toward agent-optimized web interactions enhances functionality and security in LLM-powered agent ecosystems.

7.3. Optimizing LLM multi-agent systems: Design and security

Zhou et al. [95] investigate the design complexity of Large Language Model (LLM)-powered multi-agent systems (MAS) and introduce an automated optimization framework, Multi-Agent System Search (MASS), to streamline the process. Recognizing that prompts and interaction topologies are critical for enabling effective collaboration among agents, the authors conduct a comprehensive analysis of the MAS design space. Building on these insights, they present MASS, which employs a staged optimization process that iteratively refines prompts and system topologies across three levels: block-level prompt optimization, workflow topology optimization, and global prompt optimization. Experimental results demonstrate that systems designed with MASS significantly outperform existing MAS approaches. The authors conclude by outlining key design principles for building robust, high-performing multi-agent systems.

Despite these advancements, several open challenges and vulnerabilities remain unaddressed in the design and deployment of LLM-driven MAS. The automation of prompt and topology generation, while improving system efficiency, introduces new risks, including unpredictable agent behavior, prompt injection attacks, and adversarial manipulation of communication topologies. Moreover, as MAS grow more complex, detecting unintended coordination patterns or hidden vulnerabilities becomes increasingly difficult. Future research should focus on integrating security constraints directly into the MASS optimization process, alongside the development of verification techniques to analyze the safety and robustness of optimized multi-agent configurations. Promising directions also include designing resilient topologies that can withstand adversarial disruptions, incorporating defensive prompting strategies, and exploring real-time monitoring tools to detect and mitigate malicious agent interactions in LLM-powered MAS.

7.4. VLM-powered web agents and security challenges

Andreux et al. [96] introduce Surfer-H, a cost-efficient web agent that leverages Vision–Language Models (VLMs) to perform user-defined tasks within web environments autonomously. Central to this system is Holo1, an open-weight VLM collection specifically trained for web navigation and information extraction. Holo1 is developed using a diverse dataset that combines open-access web content, synthetic examples, and self-generated agentic data. Experimental results demonstrate that Holo1 outperforms general-purpose UI benchmarks and achieves leading results on the newly proposed WebClick benchmark for web UI localization. When integrated with Holo1, Surfer-H achieves a state-of-the-art accuracy of 92.2% on the WebVoyager benchmark, offering a strong balance between task performance and cost efficiency. Therefore, several open challenges and security vulnerabilities arise when using VLM-powered web agents, such as Surfer-H. Integrating vision and language capabilities for autonomous web interaction raises concerns about interface manipulation, adversarial attacks on visual perception, and the misuse of automated browsing for malicious purposes, such as data scraping or phishing. Furthermore, the reliance on agent-generated data during training may unintentionally introduce biased behaviors or security flaws. Future research should focus on strengthening the robustness of VLM-based web agents against adversarial inputs, ensuring secure interaction with web environments, and establishing guidelines to prevent the abuse of agentic web technologies. Efforts toward explainability, real-time anomaly detection, and secure agent deployment practices will be essential for building reliable and trustworthy web agent ecosystems.

7.5. Memory-centric LLMs for long-term dialogue and emerging security risks

Chhikara et al. [97] present Mem0, a scalable memory-centric architecture designed to overcome the inherent limitations of Large Language Models (LLMs) in maintaining consistency over long, multi-session dialogues. Recognizing the challenges posed by fixed context windows, Mem0 dynamically extracts, consolidates, and retrieves relevant information from conversations to ensure contextual coherence. The authors also introduce an enhanced variant that incorporates graph-based memory structures to model complex relationships among conversational elements. Through extensive evaluations on the LO-COMO benchmark, the proposed approaches are compared against six established memory and retrieval baselines, including memory-augmented systems, RAG techniques, full-context models, and proprietary solutions. The results demonstrate that both Mem0 and its graph-based extension outperform all competing methods across diverse question categories, while significantly reducing computational overhead. Mem0 achieves notable improvements in accuracy and efficiency, highlighting the value of persistent, structured memory for advancing long-term conversational capabilities in LLM-driven agents.

While Mem0 advances the state of memory-enhanced LLMs, its reliance on structured memory mechanisms also introduces new vulnerabilities that require attention. The processes of information extraction, consolidation, and retrieval within long-term memory could be exploited by adversaries through data poisoning, manipulation of stored knowledge, or injection of misleading conversational elements. Additionally, the use of graph-based memory structures raises concerns about unauthorized inference of hidden relationships or the leakage of sensitive information. Future research should focus on securing memory retrieval pipelines, implementing tamper-resistant memory architectures, and developing mechanisms for real-time monitoring of memory integrity to ensure robustness and security. Promising directions include integrating privacy-preserving techniques, testing for adversarial robustness in memory components, and formal verification of memory consistency to ensure the effectiveness and security of LLM-powered conversational agents.

7.6. Evolutionary coding agents and emerging security challenges

Novikov et al. [98] introduce AlphaEvolve, an autonomous evolutionary coding agent designed to push the boundaries of Large Language Models (LLMs) in addressing complex scientific and engineering challenges. AlphaEvolve operates by orchestrating a pipeline of LLMs that iteratively refine source code based on evaluators' feedback. This evolutionary process enables the agent to optimize algorithms and potentially uncover novel, high-performance solutions. The authors demonstrate AlphaEvolve's broad applicability through its successful deployment on critical computational problems, including the optimization of data center scheduling algorithms, simplification of hardware accelerator circuits, and acceleration of LLM training. Impressively, AlphaEvolve also discovered provably correct algorithms that surpass existing state-of-the-art methods in both mathematics and computer science, underscoring its potential to automate scientific discovery at scale.

While AlphaEvolve represents a significant advancement in automated coding agents, its architecture also introduces new vulnerabilities and security risks that must be carefully considered. The autonomous modification of code, especially in critical systems, creates potential attack vectors, such as the introduction of subtle backdoors, the exploitation of evaluator feedback loops, or the undetected generation of unsafe code. Moreover, the iterative nature of AlphaEvolve could amplify vulnerabilities if malicious code modifications are reinforced over successive generations. Future research should prioritize the development of secure evaluation mechanisms, formal verification of generated code, and techniques for detecting adversarial manipulation within the evolutionary process. Promising directions also include implementing strict access controls, real-time monitoring, and integrating secure coding standards to ensure that autonomous coding agents, such as AlphaEvolve, contribute to scientific and engineering progress without compromising system integrity.

7.7. Robustness and scalability of defense mechanisms

Despite the proliferation of input sanitization and prompt-filtering techniques, adversaries continue to discover new injection and jail-break strategies that evade existing defenses. Current solutions often rely on static rules or model-specific heuristics, which break down when faced with adaptive attackers who leverage long contexts, compositional prompts, or multi-modal perturbations. Furthermore, most defense evaluations rely on limited benchmarks that fail to capture the full range of real-world inputs, leaving gaps in our understanding of LLM vulnerabilities.

Future work could prioritize the development of scalable, modality-agnostic defense frameworks that offer formal guarantees of robustness. This includes adversarial training regimes that integrate continuous red-teaming (e.g., jailbreak fuzzers) into the development lifecycle, formal verification methods to certify the correctness of sanitizers and filters, and cross-model transfer evaluation to ensure defenses generalize beyond a single LLM implementation. Finally, building open, extensible benchmarks encompassing text, code, image, and audio attacks will be critical for measuring progress and guiding the community toward truly resilient systems.

7.8. Securing model parameters and memory integrity

Model-compromise attacks — such as backdoors and data- or memory-poisoning — pose a serious threat because they persist beyond individual queries and can be nearly impossible to detect at inference time. Stealthy triggers embedded during fine-tuning or through external memory banks can lie dormant until activated, undermining trust in the model's outputs. Moreover, dynamic encryption techniques (e.g., DemonAgent) and composite backdoors that scatter triggers across inputs make traditional audit and anomaly detection insufficient.

Addressing these challenges requires new tools for end-to-end pipeline auditing and runtime integrity verification. Promising directions include: (1) cryptographic attestation of training artifacts and prompt templates to ensure provenance; (2) differential privacy and noise injection in memory retrieval to limit an attacker's ability to influence the agent's stored context; (3) continuous monitoring of model internals using lightweight trapdoors or canary triggers that signal unauthorized backdoor activations; and (4) tooling for transparent inspection and rollback of memory banks in deployed agents. By combining these approaches, we can begin building models and agents whose internal states are both auditable and tamper-resistant.

7.9. Future mitigation directions for MCP SDK security

Another critical open challenge concerns the secure design and deployment of Model Context Protocol (MCP) Software Development Kits (SDKs), which often serve as the backbone for agent-to-tool integrations. To enhance their robustness, developers should adopt mitigation-oriented engineering practices such as enforcing least-privilege permissions, using mutual authentication and encrypted communications, validating structured data schemas, monitoring runtime anomalies, and maintaining cryptographically verifiable provenance logs. Furthermore, sandboxing plugin execution and auditing dependencies helps prevent malicious package injection and ensures the long-term resilience of MCP-based ecosystems. These practical measures provide a roadmap for strengthening SDK security and lay the foundation for trustworthy, large-scale agentic infrastructures.

7.10. Resilience in multi-agent and federated environments

The rise of multi-agent coordination protocols (MCP, A2A) and federated learning frameworks has expanded the attack surface to include inter-agent communication channels and decentralized training pipelines. Context manipulation, contagious recursive blocking (Corba), and federated local model poisoning enable adversaries to disrupt collaborations or leak private data across agents. Side-channel and membership inference attacks further compromise confidentiality in shared environments, while social engineering simulations demonstrate that human-in-the-loop scenarios remain vulnerable.

The term *resilient multi-agent environments* refers to systems capable of maintaining functionality and coordination in the face of both accidental faults and deliberate adversarial disruptions. Specifically, resilience encompasses fault tolerance — ensuring continued operation despite node or communication failures — and adversarial robustness — preserving secure interactions even when agents are exposed to manipulation, poisoning, or collusion.

Future research could focus on designing provably secure communication and consensus protocols tailored for LLM-based agents to secure these distributed systems. This may involve: (1) integrating secure multi-party computation and threshold cryptography to protect model updates in federated training; (2) establishing cross-agent attestation mechanisms that verify the integrity and alignment of each participant before task delegation; (3) adopting blockchain or distributed ledger anchors for immutable audit trails of inter-agent messages; and (4) developing differential privacy guarantees for federated model aggregation to thwart membership inference. Together, these strategies can help ensure that agent societies and federated networks remain robust against coordinated and emerging threats.

8. Conclusion

In this paper, we presented the first unified, end-to-end threat model for LLM-powered AI agent ecosystems, spanning both host-to-tool and agent-to-agent communications. We began by formalizing adversary capabilities and attacker objectives, then systematically cataloged over thirty attack techniques across four high-level domains:

Input Manipulation, Model Compromise, System & Privacy Attacks, and Protocol Vulnerabilities. For each category, we examined representative scenarios, evaluated real-world feasibility, and reviewed existing defenses. Building on this analysis, we identified critical gaps—such as dynamic trust management for MCP, secure design of Agentic Web Interfaces, adversarial robustness in multi-agent optimization and VLM-based web agents, tamper-resistant memory architectures, secure evolutionary coding pipelines, scalable modality-agnostic defense frameworks, end-to-end integrity verification of model parameters, and resilience in federated and multi-agent environments.

Our comprehensive taxonomy and threat survey provide a foundation for designing robust defense mechanisms and establishing best practices for resilient, secure LLM-agent workflows. This framework provides practical value for developers securing agent workflows, researchers designing defenses, and policymakers establishing safety standards for LLM-based systems. We hope this work will guide future research and practical development toward closing the identified security gaps and enabling trustworthy, autonomous AI agents in real-world deployments.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the College of Information Technology, United Arab Emirates University, United Arab Emirates.

References

- [1] R. Sapkota, K.I. Roumeliotis, M. Karkee, Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenge, 2025, arXiv preprint [arXiv:2505.10468](https://arxiv.org/abs/2505.10468).
- [2] M.A. Ferrag, N. Tihanyi, M. Debbah, From llm reasoning to autonomous ai agents: A comprehensive review, 2025, arXiv preprint [arXiv:2504.19678](https://arxiv.org/abs/2504.19678).
- [3] Z. Amiri, A. Heidari, N. Jafari, M. Hosseinzadeh, Deep study on autonomous learning techniques for complex pattern recognition in interconnected information systems, *Comput. Sci. Rev.* 54 (2024) 100666.
- [4] M.A. Ferrag, N. Tihanyi, M. Debbah, Reasoning beyond limits: Advances and open problems for llms, 2025, arXiv preprint [arXiv:2503.22732](https://arxiv.org/abs/2503.22732).
- [5] A. Heidari, N.J. Navimipour, M. Unal, A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones, *IEEE Internet Things J.* 10 (10) (2023) 8445–8454.
- [6] R. Sapkota, K.I. Roumeliotis, M. Karkee, Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai, 2025, arXiv preprint [arXiv:2505.19443](https://arxiv.org/abs/2505.19443).
- [7] B. Yan, X. Zhang, L. Zhang, L. Zhang, Z. Zhou, D. Miao, C. Li, Beyond self-talk: A communication-centric survey of llm-based multi-agent systems, 2025, arXiv preprint [arXiv:2502.14321](https://arxiv.org/abs/2502.14321).
- [8] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, M. Shmueli-Scheuer, Survey on evaluation of llm-based agents, 2025, arXiv preprint [arXiv:2503.16416](https://arxiv.org/abs/2503.16416).
- [9] A. Heidari, M.A. Jabraeli Jamali, Internet of things intrusion detection systems: a comprehensive review and future directions, *Clust. Comput.* 26 (6) (2023) 3753–3780.
- [10] F. Jiang, C. Pan, L. Dong, K. Wang, O.A. Dobre, M. Debbah, From large ai models to agentic ai: A tutorial on future intelligent communications, 2025, arXiv preprint [arXiv:2505.22311](https://arxiv.org/abs/2505.22311).
- [11] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H.D. Nguyen, Multi-agent collaboration mechanisms: A survey of LLMs, 2025, arXiv preprint [arXiv:2501.06322](https://arxiv.org/abs/2501.06322).
- [12] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, Y. Xiang, Ai agents under threat: A survey of key security challenges and future pathways, *ACM Comput. Surv.* 57 (7) (2025) 1–36.
- [13] M.L. Romero, R. Suyama, Agentic AI for intent-based industrial automation, 2025, arXiv preprint [arXiv:2506.04980](https://arxiv.org/abs/2506.04980).
- [14] Y. Yigit, M.A. Ferrag, M.C. Ghanem, I.H. Sarker, L.A. Maglaras, C. Chrysoulas, N. Moradpoor, N. Tihanyi, H. Janicke, Generative AI and LLMs for critical infrastructure protection: evaluation benchmarks, agentic AI, challenges, and opportunities, *Sensors* 25 (6) (2025) 1666.

- [15] V.S. Narajala, I. Habler, Enterprise-grade security for the model context protocol (mcp): Frameworks and mitigation strategies, 2025, arXiv preprint [arXiv:2504.08623](#).
- [16] A. Ehteshami, A. Singh, G.K. Gupta, S. Kumar, A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp), 2025, arXiv preprint [arXiv:2505.02279](#).
- [17] Y. Yang, H. Chai, Y. Song, S. Qi, M. Wen, N. Li, J. Liao, H. Hu, J. Lin, G. Chang, et al., A survey of AI agent protocols, 2025, arXiv preprint [arXiv:2504.16736](#).
- [18] J. Chen, J. Ye, G. Wang, From standalone LLMs to integrated intelligence: A survey of compound AI systems, 2025, arXiv preprint [arXiv:2506.04565](#).
- [19] H. Wei, D.Y. Lee, S. Rohal, Z. Hu, S. Fang, S. Pan, A survey of foundation models for IoT: Taxonomy and criteria-based analysis, 2025, arXiv preprint [arXiv:2506.12263](#).
- [20] Q. Zhan, R. Fang, H.S. Panchal, D. Kang, Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents, 2025, arXiv preprint [arXiv:2503.00061](#).
- [21] S. Jiang, X. Chen, R. Tang, Deceiving LLM through compositional instruction with hidden attacks, *ACM Trans. Auton. Adapt. Syst.* (2025).
- [22] J. Yu, X. Lin, Z. Yu, X. Xing, Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts, 2023, arXiv preprint [arXiv:2309.10253](#).
- [23] D. Schwartz, D. Bespalov, Z. Wang, N. Kulkarni, Y. Qi, Graph of attacks with pruning: Optimizing stealthy jailbreak prompt generation for enhanced LLM content moderation, 2025, arXiv preprint [arXiv:2501.18638](#).
- [24] Y. Chen, X. Hu, K. Yin, J. Li, S. Zhang, Aelia-mn: Evaluating the robustness of multimodal LLM-powered mobile agents against active environmental injection attacks, 2025, arXiv preprint [arXiv:2502.13053](#).
- [25] H. Huang, Z. Zhao, M. Backes, Y. Shen, Y. Zhang, Composite backdoor attacks against large language models, 2023, arXiv preprint [arXiv:2310.07676](#).
- [26] P. Zhu, Z. Zhou, Y. Zhang, S. Yan, K. Wang, S. Su, Demonagent: Dynamically encrypted multi-backdoor implantation attack on LLM-based agent, 2025, arXiv preprint [arXiv:2502.12575](#).
- [27] W. Zou, R. Geng, B. Wang, J. Jia, Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models, 2024, arXiv preprint [arXiv:2402.07867](#).
- [28] A. Heidari, N. Jafari Navimipour, M.A. Jabraeil Jamali, S. Akbarpour, Securing and optimizing IoT offloading with blockchain and deep reinforcement learning in multi-user environments, *Wirel. Netw.* 31 (4) (2025) 3255–3276.
- [29] A. Heidari, N.J. Navimipour, S. Zeadally, V. Chamola, Everything you wanted to know about ChatGPT: Components, capabilities, applications, and opportunities, *Internet Technol. Lett.* 7 (6) (2024) e530.
- [30] C. Zhang, Y. Deng, X. Lin, B. Wang, D. Ng, H. Ye, X. Li, Y. Xiao, Z. Mo, Q. Zhang, et al., 100 days after deepseek-r1: A survey on replication studies and more directions for reasoning language models, 2025, arXiv preprint [arXiv:2505.00551](#).
- [31] X. Hou, Y. Zhao, S. Wang, H. Wang, Model context protocol (mcp): Landscape, security threats, and future research directions, 2025, arXiv preprint [arXiv:2503.23278](#).
- [32] Z. Yang, W. Xu, L. Liang, et al., On privacy, security, and trustworthiness in distributed wireless large AI models, *Sci. China Inf. Sci.* 68 (2025) 170301, <http://dx.doi.org/10.1007/s11432-024-4465-3>.
- [33] M. Asadi, A. Heidari, N. Jafari Navimipour, A new flow-based approach for enhancing botnet detection efficiency using convolutional neural networks and long short-term memory, *Knowl. Inf. Syst.* 67 (7) (2025) 6139–6170.
- [34] K. Wang, G. Zhang, Z. Zhou, J. Wu, M. Yu, S. Zhao, C. Yin, J. Fu, Y. Yan, H. Luo, et al., A comprehensive survey in LLM (-agent) full stack safety: Data, training and deployment, 2025, arXiv preprint [arXiv:2504.15585](#).
- [35] A. Chen, Y. Wu, J. Zhang, S. Yang, J.-t. Huang, K. Wang, W. Wang, S. Wang, A survey on the safety and security threats of computer-using agents: JARVIS or ultron?, 2025, arXiv preprint [arXiv:2505.10924](#).
- [36] O. Friha, M.A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, N. Ghoualmi-Zine, LLM-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness, *IEEE Open J. Commun. Soc.* (2024).
- [37] M.A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, M. Debbah, Generative AI in cybersecurity: A comprehensive review of LLM applications and vulnerabilities, *Internet Things Cyber-Physical Syst.* (2025).
- [38] Y. Yang, R. Gao, X. Wang, T.-Y. Ho, N. Xu, Q. Xu, Mma-diffusion: Multimodal attack on diffusion models, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7737–7746.
- [39] Y. Li, G. Liu, Y. Yang, C. Wang, Seeing is believing: Black-box membership inference attacks against retrieval augmented generation, 2024, pp. arXiv-2406, arXiv e-prints.
- [40] F. Perez, I. Ribeiro, Ignore previous prompt: Attack techniques for language models, 2022, arXiv preprint [arXiv:2211.09527](#).
- [41] R. Pedro, D. Castro, P. Carreira, N. Santos, From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?, 2023, arXiv preprint [arXiv:2308.01990](#).
- [42] E. Bagdasaryan, T.-Y. Hsieh, B. Nassi, V. Shmatikov, Abusing images and sounds for indirect instruction injection in multi-modal LLMs, 2023, arXiv preprint [arXiv:2307.10490](#).
- [43] Z. Wei, Y. Wang, A. Li, Y. Mo, Y. Wang, Jailbreak and guard aligned language models with only few in-context demonstrations, 2023, arXiv preprint [arXiv:2310.06387](#).
- [44] E. Shayegani, Y. Dong, N. Abu-Ghazaleh, Jailbreak in pieces: Compositional adversarial attacks on multi-modal language models, 2023, arXiv preprint [arXiv:2307.14539](#).
- [45] C. Anil, E. Durmus, N. Panickssery, M. Sharma, J. Benton, S. Kundu, J. Batson, M. Tong, J. Mu, D. Ford, et al., Many-shot jailbreaking, *Adv. Neural Inf. Process. Syst.* 37 (2024) 129696–129742.
- [46] X. Liu, N. Xu, M. Chen, C. Xiao, Autodan: Generating stealthy jailbreak prompts on aligned large language models, 2023, arXiv preprint [arXiv:2310.04451](#).
- [47] Y. Dong, X. Meng, N. Yu, Z. Li, S. Guo, Fuzz-testing meets LLM-based agents: An automated and efficient framework for jailbreaking text-to-image generation models, in: *2025 IEEE Symposium on Security and Privacy, SP, IEEE Computer Society*, 2025, pp. 336–354.
- [48] J. Wang, Z. Liu, K.H. Park, Z. Jiang, Z. Zheng, Z. Wu, M. Chen, C. Xiao, Adversarial demonstration attacks on large language models, 2023, arXiv preprint [arXiv:2305.14950](#).
- [49] H. Zhuang, Y. Zhang, S. Liu, A pilot study of query-free adversarial attack against stable diffusion, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 2385–2392.
- [50] A.S. Patlan, P. Sheng, S.A. Hebbat, P. Mittal, P. Viswanath, Real AI agents with fake memories: Fatal context manipulation attacks on Web3 agents, 2025, arXiv preprint [arXiv:2503.16248](#).
- [51] M. Milanta, L. Beurer-Kellner, GitHub MCP exploited: Accessing private repositories via MCP, 2025, [Online]. Available: <https://invariantlabs.ai/blog/mcp-github-vulnerability>. (Accessed 22 November 2025).
- [52] W. Yang, X. Bi, Y. Lin, S. Chen, J. Zhou, X. Sun, Watch out for your agents! Investigating backdoor threats to LLM-based agents, *Adv. Neural Inf. Process. Syst.* 37 (2024) 100938–100964.
- [53] X. Cai, H. Xu, S. Xu, Y. Zhang, et al., Badprompt: Backdoor attacks on continuous prompts, *Adv. Neural Inf. Process. Syst.* 35 (2022) 37068–37080.
- [54] H. Yao, J. Lou, Z. Qin, Poisonprompt: Backdoor attack on prompt-based large language models, in: *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE*, 2024, pp. 7745–7749.
- [55] Y. Wang, D. Xue, S. Zhang, S. Qian, Badagent: Inserting and activating backdoor attacks in LLM agents, 2024, arXiv preprint [arXiv:2406.03007](#).
- [56] D.A. Alber, Z. Yang, A. Alyakin, E. Yang, S. Rai, A.A. Valliani, J. Zhang, G.R. Rosenbaum, A.K. Amend-Thomas, D.B. Kurland, et al., Medical large language models are vulnerable to data-poisoning attacks, *Nature Med.* (2025) 1–9.
- [57] Q. Zhang, B. Zeng, C. Zhou, G. Go, H. Shi, Y. Jiang, Human-imperceptible retrieval poisoning attacks in LLM-powered applications, in: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 502–506.
- [58] E. Wallace, T.Z. Zhao, S. Feng, S. Singh, Concealed data poisoning attacks on NLP models, 2020, arXiv preprint [arXiv:2010.12563](#).
- [59] M. Fang, X. Cao, J. Jia, N. Gong, Local model poisoning attacks to {byzantine-robust} federated learning, in: *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1605–1622.
- [60] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, Z. Xiang, A practical memory injection attack against LLM agents, 2025, arXiv preprint [arXiv:2503.03704](#).
- [61] S. Han, B. Buyukates, Z. Hu, H. Jin, W. Jin, L. Sun, X. Wang, W. Wu, C. Xie, Y. Yao, et al., Fedsecurity: Benchmarking attacks and defenses in federated learning and federated LLMs, 2023, arXiv preprint [arXiv:2306.04959](#).
- [62] Z. Qi, H. Zhang, E. Xing, S. Kakade, H. Lakkaraju, Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems, 2024, arXiv preprint [arXiv:2402.17840](#).
- [63] M. Soleimani, G. Jia, I. Gim, S.-s. Lee, A. Khandelwal, Wiretapping LLMs: Network side-channel attacks on interactive LLM services, *Cryptol. EPrint Arch.* (2025).
- [64] Z. Zhou, Z. Li, J. Zhang, Y. Zhang, K. Wang, Y. Liu, Q. Guo, CORBA: Contagious recursive blocking attacks on multi-agent systems based on large language models, 2025, arXiv preprint [arXiv:2502.14529](#).
- [65] T. Kumarage, C. Johnson, J. Adams, L. Ai, M. Kirchner, A. Hoogs, J. Garland, J. Hirschberg, A. Basharat, H. Liu, Personalized attacks of social engineering in multi-turn conversations–LLM agents for simulation and detection, 2025, arXiv preprint [arXiv:2503.15552](#).
- [66] Introduction to MCP, 2025, [Online]. Available: <https://modelcontextprotocol.io/introduction>. (Accessed: 04 June 2025).
- [67] A2A: A New Era of Agent Interoperability, 2025, [Online]. Available: <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>. (Accessed 04 June 2025).
- [68] Agent network protocol GitHub repository, 2024, <https://github.com/agent-network-protocol/AgentNetworkProtocol>. (Accessed June 2025).
- [69] Agent communication protocol, 2024, <https://agentcommunicationprotocol.dev/introduction/welcome>. (Accessed June 2025).
- [70] I. Habler, K. Huang, V.S. Narajala, P. Kulkarni, Building a secure agentic AI application leveraging A2A protocol, 2025, arXiv preprint [arXiv:2504.16902](#).
- [71] CVE-2025-46059: LangChain – GmailToolkit (v0.3.51) Indirect Prompt Injection Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-46059>. (Accessed October 2025).

- [72] CWE-94: Improper Control of Generation of Code ('Code Injection'), 2025, <https://cwe.mitre.org/data/definitions/94.html>. (Accessed October 2025).
- [73] CVE-2025-2828: LangChain – RequestsToolkit (OpenAPI Tool) SSRF Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-2828>. (Accessed October 2025).
- [74] CWE-918: Server-Side Request Forgery (SSRF), 2025, <https://cwe.mitre.org/data/definitions/918.html>. (Accessed October 2025).
- [75] CVE-2025-6984: LangChain – EverNoteLoader (v0.3.63) XML External Entity Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6984>. (Accessed October 2025).
- [76] CWE-200: Exposure of Sensitive Information to an Unauthorized Actor, 2025, <https://cwe.mitre.org/data/definitions/200.html>. (Accessed October 2025).
- [77] CVE-2025-6985: LangChain – HTMLSectionSplitter (v0.3.8) XSLT Injection Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6985>. (Accessed October 2025).
- [78] CWE-611: Improper Restriction of XML External Entity Reference ('XXE'), 2025, <https://cwe.mitre.org/data/definitions/611.html>. (Accessed October 2025).
- [79] CVE-2025-6853: LangChain-Chatchat (v0.3.1) Path Traversal in Temporary Uploads, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6853>. (Accessed October 2025).
- [80] CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'), 2025, <https://cwe.mitre.org/data/definitions/22.html>. (Accessed October 2025).
- [81] CVE-2025-6854: LangChain-Chatchat (v0.3.1) Base64 Path Traversal Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6854>. (Accessed October 2025).
- [82] CVE-2025-6855: LangChain-Chatchat (v0.3.1) Path Traversal in File Uploads, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6855>. (Accessed October 2025).
- [83] CVE-2025-11844: Hugging Face – SmolAgents (v1.20.0) XPath Injection Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-11844>. (Accessed October 2025).
- [84] CWE-643: Improper Neutralization of Data within XPath Expressions ('XPath Injection'), 2025, <https://cwe.mitre.org/data/definitions/643.html>. (Accessed October 2025).
- [85] CVE-2025-59532: OpenAI – Codex CLI (v0.2.0–0.38.0) Sandbox Escape Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-59532>. (Accessed October 2025).
- [86] CWE-20: Improper Input Validation, 2025, <https://cwe.mitre.org/data/definitions/20.html>. (Accessed October 2025).
- [87] CVE-2025-6514: Anthropic – MCP-Remote (v0.0.5–0.1.15) Command Execution Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-6514>. (Accessed October 2025).
- [88] CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection'), 2025, <https://cwe.mitre.org/data/definitions/78.html>. (Accessed October 2025).
- [89] CVE-2025-49596: Anthropic – MCP Inspector (v0.14.0) CSRF/DNS-Rebinding Vulnerability, 2025, <https://nvd.nist.gov/vuln/detail/CVE-2025-49596>. (Accessed October 2025).
- [90] CWE-306: Missing Authentication for Critical Function, 2025, <https://cwe.mitre.org/data/definitions/306.html>. (Accessed October 2025).
- [91] M.M. Hasan, H. Li, E. Fallahzadeh, B. Adams, A.E. Hassan, Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers, 2025, arXiv preprint [arXiv:2506.13538](https://arxiv.org/abs/2506.13538).
- [92] A. Ahmadi, S. Sharif, Y.M. Banad, MCP bridge: A lightweight, LLM-agnostic restful proxy for model context protocol servers, 2025, arXiv preprint [arXiv:2504.08999](https://arxiv.org/abs/2504.08999).
- [93] M. Al-Hawawreh, M. Alazab, M.A. Ferrag, M.S. Hossain, Securing the industrial internet of things against ransomware attacks: A comprehensive analysis of the emerging threat landscape and detection mechanisms, *J. Netw. Comput. Appl.* 223 (2024) 103809.
- [94] X.H. Lü, G. Kamath, M. Mosbach, S. Reddy, Build the web for agents, not agents for the web, 2025, arXiv preprint [arXiv:2506.10953](https://arxiv.org/abs/2506.10953).
- [95] H. Zhou, X. Wan, R. Sun, H. Palangi, S. Iqbal, I. Vulić, A. Korhonen, S.Ö. Arik, Multi-agent design: Optimizing agents with better prompts and topologies, 2025, arXiv preprint [arXiv:2502.02533](https://arxiv.org/abs/2502.02533).
- [96] M. Andreux, B.B. Skuk, H. Bencheikroun, E. Biré, A. Bonnet, R. Bordie, M. Brunel, P.-L. Cedoz, A. Chassang, M. Chen, et al., Surfer-h meets Holo1: Cost-efficient web agent powered by open weights, 2025, arXiv preprint [arXiv:2506.02865](https://arxiv.org/abs/2506.02865).
- [97] P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav, Mem0: Building production-ready ai agents with scalable long-term memory, 2025, arXiv preprint [arXiv:2504.19413](https://arxiv.org/abs/2504.19413).
- [98] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A.Z. Wagner, S. Shirobokov, B. Kozlovskii, F.J. Ruiz, A. Mehrabian, et al., AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025, arXiv preprint [arXiv:2506.13131](https://arxiv.org/abs/2506.13131).